



Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

Szikszí FM

Készítették:

Szabó Dániel,
Csépányi Gergely

Ózd, 2026

Tartalom

Bevezetés	3
Felhasználói dokumentáció	3
Rendszerkövetelmény	3
Hardver:.....	3
Szoftver:	3
Webalkalmazás indítása.....	4
Webalkalmazás használata	4
Általános információk a weboldalról	4
Főoldal leírás + képernyőképek.....	5
Bejelentkezés/regisztráció oldal leírás + képernyőképek.....	6
Műsorok oldal leírás + képernyőképek.....	7
Műsorvezetők oldal leírás + képernyőképek	7
Rólunk oldal leírás + képernyőképek.....	8
Fejlesztői dokumentáció	9
Alkalmazott fejlesztői és csoportmunka eszközök	9
Adatbázis.....	9
Frontend.....	9
Backend.....	9
Adatbázis.....	10
Modell leírása(adatmezők - típusok - leírásuk)	10
Táblák, kapcsolatok	11
E-K diagramm	17
Frontend	18
Alkalmazás frontend részének részletes leírása + képernyőfotók	18
Backend	32
Alkalmazás backend részének részletes leírása + képernyőfotók	32

Bevezetés

A projekt célja egy **komplex weboldal** létrehozása az **iskolarádió** számára, amely modern, könnyen kezelhető és informatív felületet biztosít mind a diákok, mind a tanárok számára. Az oldal célja, hogy digitális platformot kínáljon az iskolai rádióműsorok, hírek, események és archív tartalmak bemutatására, ezáltal növelve a közösségi élet aktivitását és átláthatóságát.

A weboldal nem csupán információs felületként szolgál, hanem interaktív funkciókat is kínál majd — például műsorrend megtekintését, élő adások hallgatását, valamint a diákok által készített tartalmak közzétételét. A projekt során a tervezés, fejlesztés és tesztelés lépései is megvalósulnak, különös figyelmet fordítva a **felhasználói élményre** a **reszponzív dizájnra**, valamint a **biztonságos és megbízható működésre**.

<https://github.com/szaboaprofi/Szikszi-FM.git>

Felhasználói dokumentáció

Rendszerkövetelmény

Hardver:

- Asztali gép/laptop: Legalább 4GB RAM, 2 magos CPU, internetkapcsolat
- Mobil eszköz: Android 8.0+ vagy iOS 12.0+ támogatása

Szoftver:

- VS Code v1.96.0 vagy újabb
- Node.js v22.13.0 vagy újabb
- XAMPP 8.1.12 vagy újabb
- Composer 2.8.4 vagy újabb

Webalkalmazás indítása

A webalkalmazás elindításához elkell indítanunk a XAMPP alkalmazást és elindítani az Apache illetve MySQL részét. Ezt követően meg kell nyitni egy Visual Studio Code alkalmazást és megnyitni a BACKEND mappát benne, majd CTRL+Shift+ö billentyű kombináció lenyomásával előhozzuk a terminált ennek a PowerShell részében kell kiadnunk a parancsot ami létrehozza, illetve feltölti az adatbázis: `php artisan migrate --seed`. Így az adatbázisunkat létrehoztuk már csak el kell indítani a szerveret amit a következő paranccsal tudunk végrehajtani: `php artisan serve`

Amint ezzel kész vagyunk nyitni kell még egy Visual Studio Code alkalmazást és megnyitni a FRONTEND mappát. Szintén nyitni kell egy terminált a CTRL+Shift+ö billentyű kombinációval, majd ki kell adni az `npm install` parancsot a terminal cmd részében, amikor a terminal befejezi a folyamatot ki kell adnunk a következő parancsot: `ng s -o` majd az alábbi linket bemásolni a böngésző címsorába: <https://localhost:4200/>

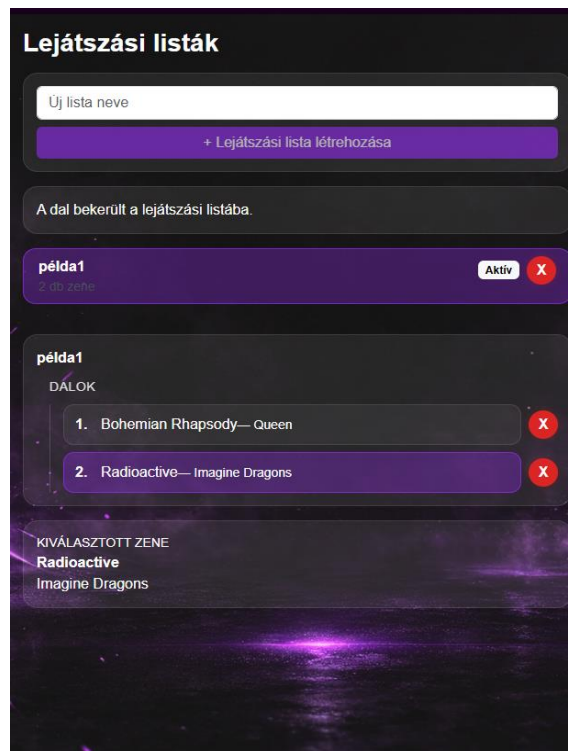
Webalkalmazás használata

Általános információk a weboldalról

Az oldalunk egy iskolarádió gördülékenyebb működését segíti elő, melyben a bejelentkezett felhasználók láthatják a heti lejátszott zenéket illetve aktuális műsorvezetőket. Ezen felül saját lejátszási listát hozhatnak létre.

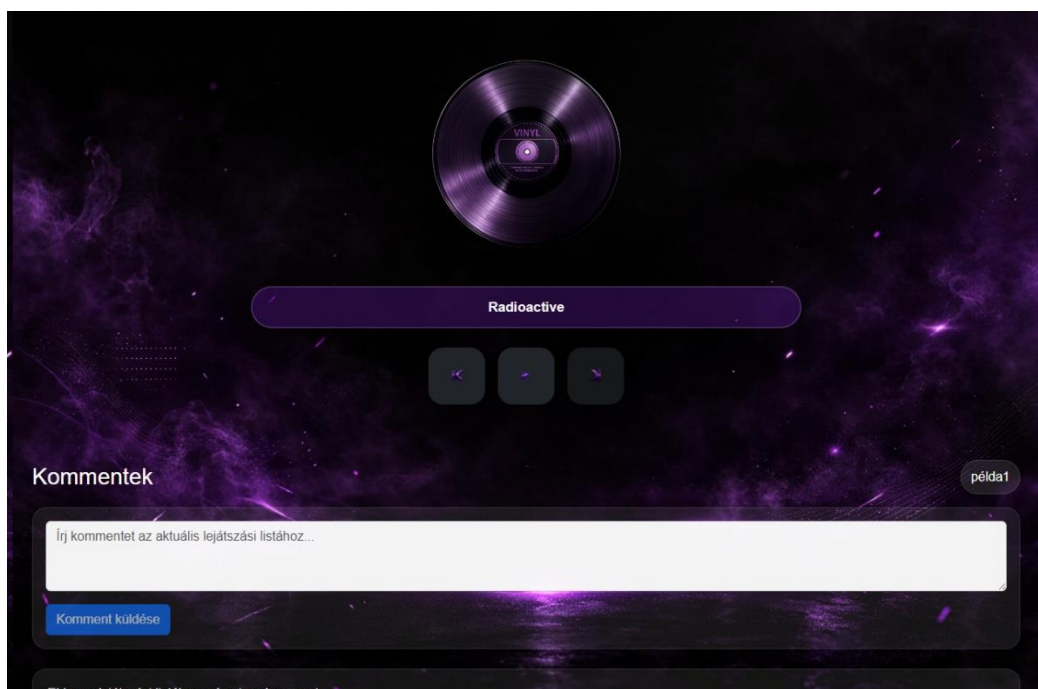
Főoldal leírás + képernyőképek

A főoldalt 3 részre lehet osztani, van a baloldali rész ahol a lejátszási listákat tudunk létrehozni illetve látjuk a már létrehozott lejátszási listáinkat



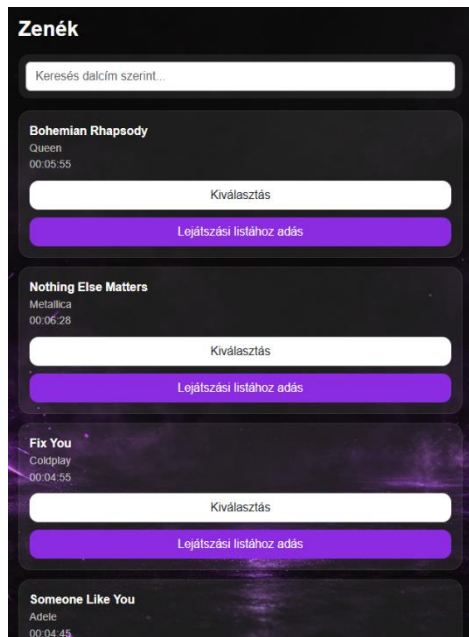
1. kép: Főoldal, lejátszási listák oszlop

Középen található a zene lejátszó illetve alatta a lejátszási listák komment felülete itt tudunk lépkedni a lejátszási lista zenéi között a jobb illetve bal nyíllal.



2. Kép: Főoldal, középső rész

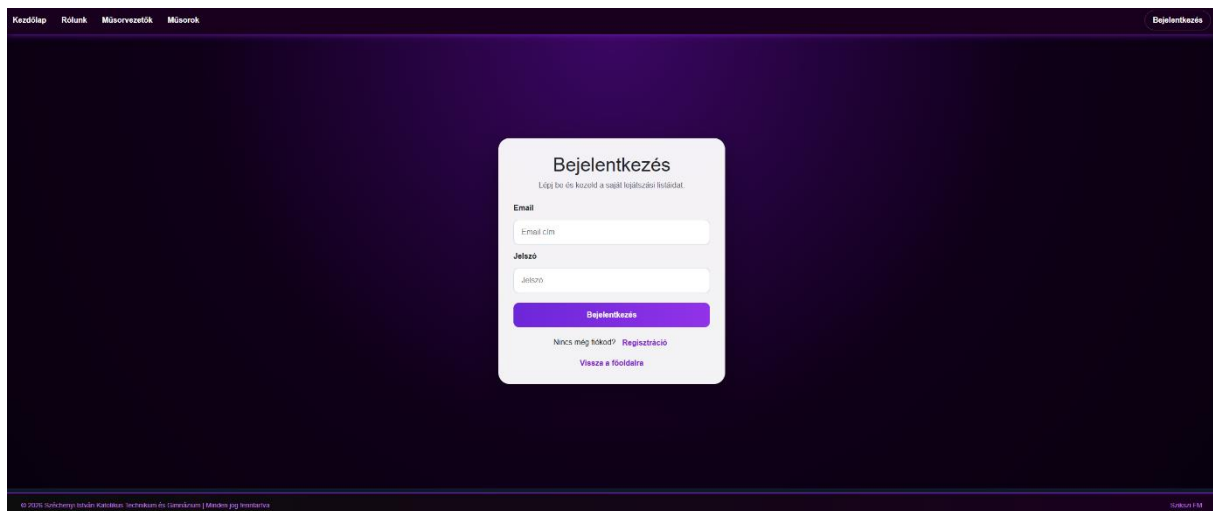
Jobboldalt pedig a zenéket látjuk amelyeket hozzáadhatunk lejátszási listához, a felső keresősávban pedig zene cím alapján kereshetünk zenékre.



3. Kép: Főoldal, Jobboldali zenék oszlop

Bejelentkezés/regisztráció oldal leírás + képernyőképek

A bejelentkezés oldalon jelentkezhetünk be fiókunkba amennyiben rendelkezünk fiókkal, ha nem egy gombnyomással válthatunk a regisztráció oldalra, ahol automatikusan bejelentkeztet minket az oldala regisztrációt követően.



4. Kép: Bejelentkezés fül

5. Kép: Regisztráció fül

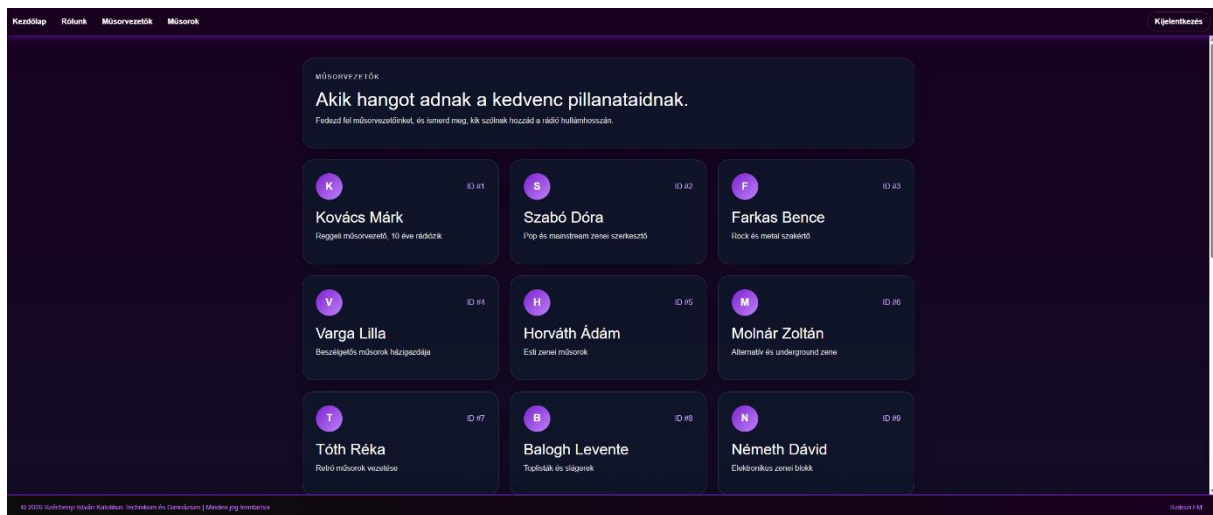
Műsorok oldal leírás + képernyőképek

A műsorok oldalon láthatjuk a heti műsorlistát, hogy ki volt/lesz az adott nap műsorvezetője, melyik zenék voltak/lesznek aznap.

6. Kép: Műsorok oldal

Műsorvezetők oldal leírás + képernyőképek

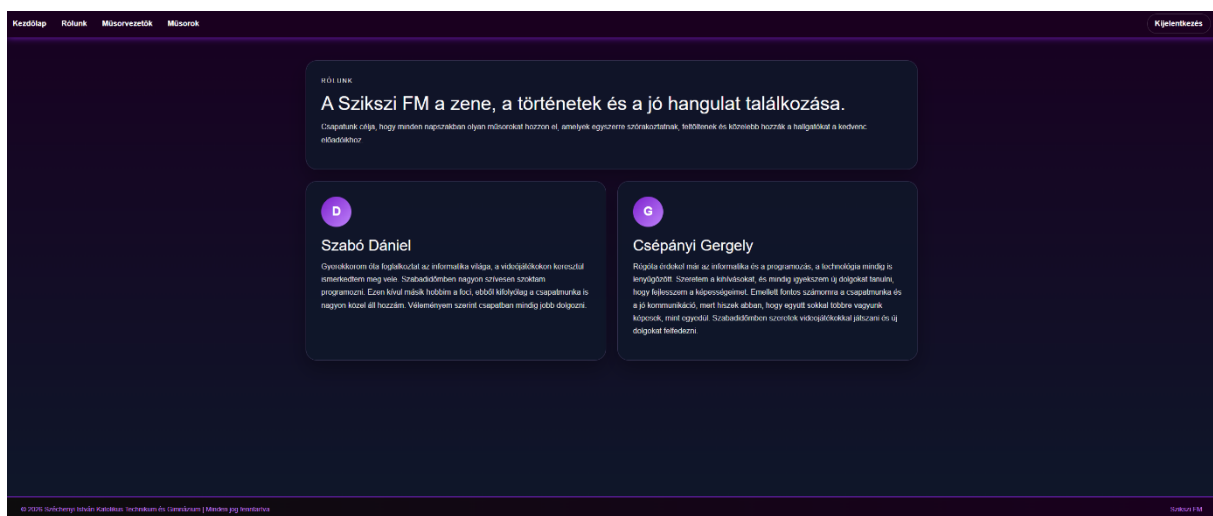
A műsorvezetők oldalon láthatjuk a műsorvezetőket, és az általuk vezetett műsorokat.



7. Kép: műsorvezetők oldal

Rólunk oldal leírás + képernyőképek

arólunk oldalon lehet rólunk, a fejlesztőkről egy rövid bemutatkozást olvasni.



8. Kép: Rólunk oldal

Fejlesztői dokumentáció

Alkalmazott fejlesztői és csoportmunka eszközök

Git/GitHub: Project tárolása és verziómenedzselés

Discord: Kommunikáció

Trello: Feladatok elosztása a csapattagok között

Adatbázis

Visual Studio Code: SQL kód megírása

XAMPP: SQL kód tesztelése, Apache webszerver futtatása

MySQL: adatbázis tárolása

Frontend

Visual Studio Code: A weboldal kódjának megírása

Angular: Keretrendszer a weboldalhoz

Figma: A weboldal megtervezése

Backend

Visual Studio Code: Laravel api, kontrollerek megírása és tesztelése

XAMPP: Adatbázis lokális futtatása és kezelése

Laravel: Keretrendszer az apihoz

Adatbázis

Modell leírása(adatmezők - típusok - leírásuk)

A feladat egy **adatbázis létrehozása egy webalkalmazáshoz** volt. A munka folyamata három fő lépésből állt:

1. **Adatbázis megtervezése** – először felmértük, milyen adatokra van szükség a webalkalmazás működéséhez, és meghatároztuk a táblák szerkezetét, a mezőket és a kapcsolódásokat.
2. **Adatbázis elkészítése** – a tervek alapján létrehoztuk a táblákat, beállítottuk az elsődleges és idegen kulcsokat, valamint feltöltöttük az adatokat tesztcélokra.
3. **EK-diagram készítése** – a kapcsolatok és struktúrák átláthatósága érdekében elkészítettük az **entitás–kapcsolat diagramot**, amely vizuálisan szemlélteti az adatbázis szerkezetét és a táblák közötti kapcsolatokat. Ez a folyamat biztosítja, hogy az adatbázis jól szervezett, konzisztens és könnyen kezelhető legyen a webalkalmazás számára.

Táblák, kapcsolatok

Felhasználók

Mező	Leírás
id	Egyedi azonosító (PK)
felhasznalonev	Felhasználónév
email	Email cím
jelszo_hash	Titkosított jelszó
szerep	Felhasználói szerepkör
letrehozva	Regisztráció dátuma

A felhasználók tábla tárolja az összes rendszerben regisztrált személy adatait. Ez az alap tábla, amelyhez több másik tábla idegen kulccsal kapcsolódik.

Műsorvezetők

Mező	Leírás
id	Egyedi azonosító (PK)
nev	Műsorvezető neve
bemutakozas	Rövid bemutatkozás
felhasznalo_id	Kapcsolódó felhasználó (FK)

A műsorvezetők a felhasználók egy speciális csoportját alkotják. Minden műsorvezetőhöz tartozik egy felhasználói fiók.

Műsorok

Mező	Leírás
id	Egyedi azonosító (PK)
cim	A műsor címe
leiras	A műsor leírása
musorvezeto_id	Műsorvezető azonosítója (FK)
letrehozva	Létrehozás dátuma

A műsorok tábla tartalmazza az egyes rádióműsorokat, amelyeket egy adott műsorvezető vezet.

Dalok

Mező	Leírás
id	Egyedi azonosító (PK)
eloado	Előadó neve
cim	Dal címe
hossza	Dal időtartama

A dalok tábla a lejátszható zeneszámokat tartalmazza, amelyek több műsorban is felhasználhatók.

Lejátszási lista

Mező	Leírás
id	Egyedi azonosító (PK)
dal_id	Dal azonosítója (FK)
sorrend_szam	Lejátszási sorrend
musor_id	Műsor azonosítója (FK)

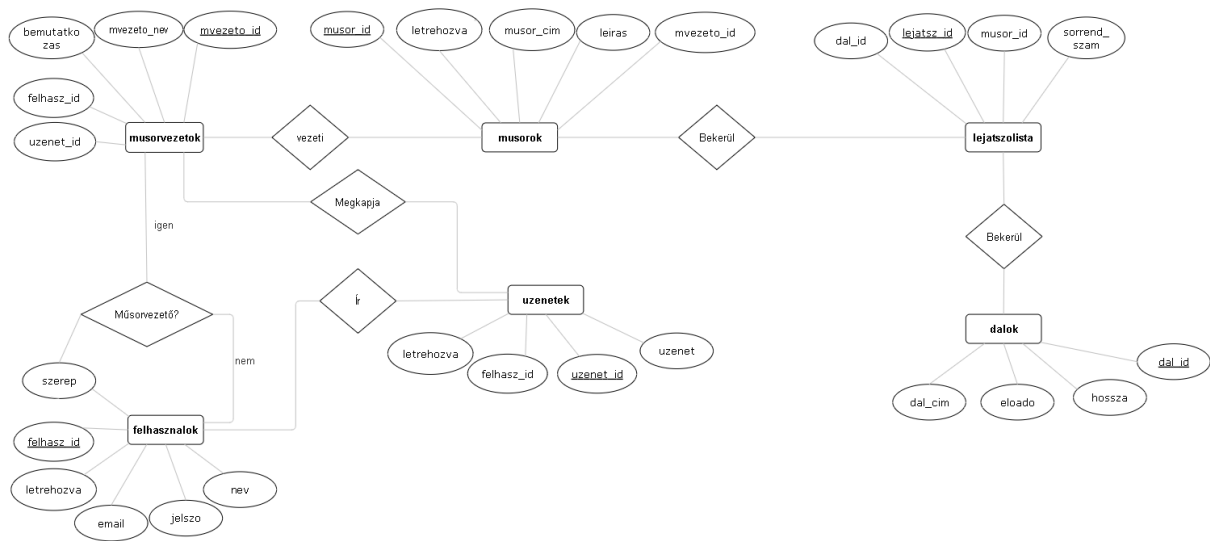
A lejátszási lista kapcsolótábla, amely meghatározza, hogy egy műsorban milyen dalok és milyen sorrendben hangzanak el.

Üzenetek

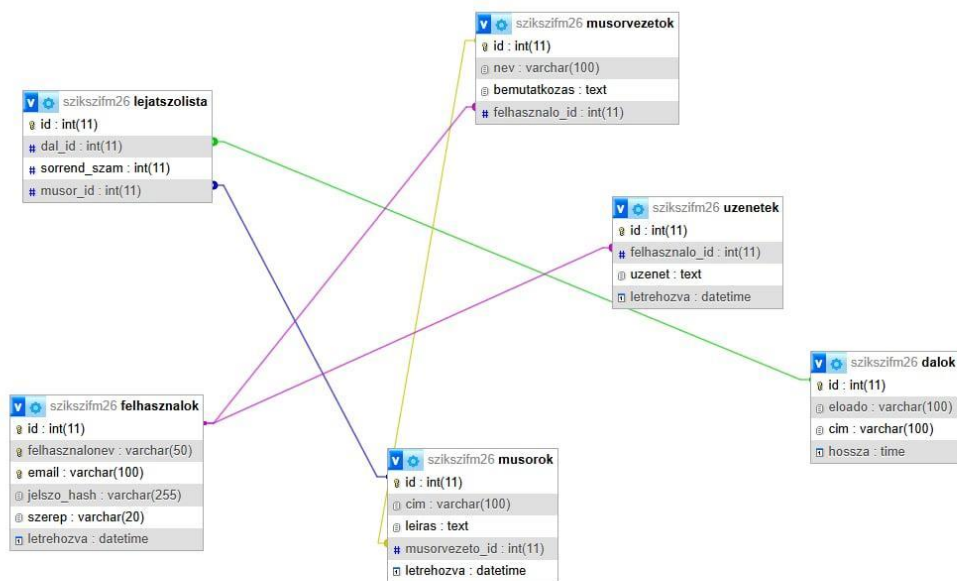
Mező	Leírás
id	Egyedi azonosító (PK)
felhasznalo_id	Felhasználó azonosítója (FK)
uzenet	Az üzenet szövege
letrehozva	Üzenet küldésének ideje

Az üzenetek tábla a hallgatók visszajelzéseit és hozzászólásait tárolja.

E-K diagramm



9. Kép:EK Diagramm



10. Kép:Adatbázis szerkezet

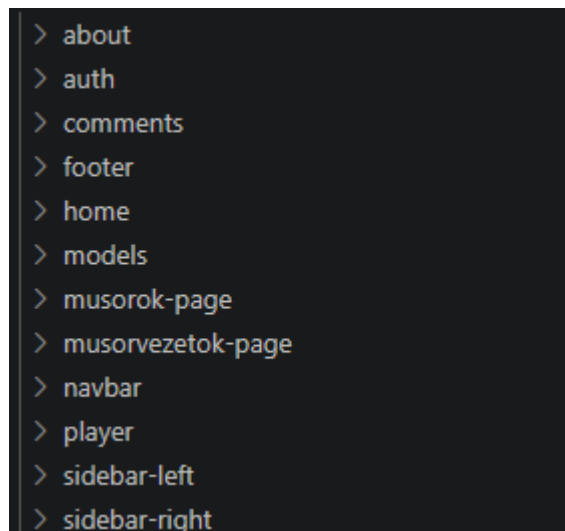
Frontend

Alkalmazás frontend részének részletes leírása + képernyőfotók

Bevezetés

A dokumentáció ezen része a webalkalmazás frontend részének működését és felépítését mutatja be. A frontend az alkalmazás azon része, amely közvetlen kapcsolatban áll a felhasználóval, tehát ez biztosítja a grafikus felületet, az interakciókat és az adatok megjelenítését. A frontend fő feladata, hogy a backendből érkező adatokat feldolgozza és megjelenítse, valamint a felhasználói műveleteket (például kattintások, bevitel, navigáció) kezelje. Emellett biztosítja az alkalmazás állapotának kezelését és a különböző nézetek közötti váltást. Az alkalmazás Angular keretrendszerrel készült, amely lehetővé teszi a komponens alapú felépítést. Ez azt jelenti, hogy a felhasználói felület kisebb, egymástól elkülönített részekből épül fel, amelyek külön-külön is kezelhetők és újra felhasználhatók. A projekt modern Angular megoldásokat alkalmaz, például standalone komponenseket és signal alapú állapotkezelést.

Komponens alapú felépítés



11. Kép: Komponensek

A frontend több különálló komponensből épül fel, amelyek mindegyike egy adott funkcióért felel. Ez a megközelítés lehetővé teszi a kód átláthatóbbá tételét és a fejlesztés egyszerűsítését. Minden komponens tartalmaz egy HTML sablont, amely a megjelenítésért felel, valamint egy TypeScript fájlt, amely a működés logikáját tartalmazza. A komponensek közötti kommunikáció jellemzően service-eken keresztül történik, amelyek az adatkezelést végzik.

Fő alkalmazás

```
<div class="app-shell">
  <app-navbar></app-navbar>

  <main class="app-main">
    <router-outlet></router-outlet>
  </main>

  <app-footer></app-footer>
</div>
```

12. Kép: App komponens

Az alkalmazás központi eleme az App komponens, amely az egész felület vázát biztosítja. Ez a komponens tartalmazza a navigációs sávot, az aktuális oldalt megjelenítő részt és a lábléceket. A navigációs sáv minden oldalon látható, és lehetővé teszi a felhasználó számára az oldalak közötti váltást. A RouterOutlet nevű elem felel azért, hogy az aktuális URL alapján a megfelelő komponens jelenjen meg. A lábléc az oldal alján helyezkedik el, és általában statikus információkat tartalmaz.

Home oldal működése

```
<div class="container-fluid main-container">
  <div class="row h-100">
    <div class="col-12 col-lg-3 p-0 center-scroll position-relative">
      <app-sidebar-left></app-sidebar-left>
    </div>

    <div class="col-12 col-lg-6 p-0 center-scroll position-relative">
      <app-player></app-player>
      <app-comments></app-comments>
    </div>

    <div class="col-12 col-lg-3 p-0 center-scroll position-relative">
      <app-sidebar-right></app-sidebar-right>
    </div>
  </div>
</div>
```

13. Kép: Home oldal

A Home oldal az alkalmazás központi része, ahol a legtöbb funkció elérhető. Ez az oldal több kisebb komponensből áll össze, amelyek együtt biztosítják a teljes funkcionalitást. A Home oldal figyeli a bejelentkezett felhasználó állapotát, és ennek megfelelően tölti be a szükséges adatokat, például a felhasználóhoz tartozó playlisteket. Ha a felhasználó változik, az oldal automatikusan frissíti a megjelenített adatokat.

Az oldal felépítésé több részre osztható. A bal oldalon a playlist kezelés található, a jobb oldalon a zenék listája jelenik meg, középen a lejátszó, alul pedig a kommentek.

Playlist kezelés (SidebarLeft)

```
1 <div class="sidebar-left p-3 h-100">
2   <h3 class="fw-bold mb-3">Lejátszási listák</h3>
3   @if (!user()) {
4     <p class="hint">Jelentkezz be, hogy saját listákat hozhass létre.</p>
5   } @else {
6     <div class="create-card mb-3">
7       <input
8         type="text"
9         class="form-control mb-2"
10        [ngModel]="newPlaylistName()"
11        (ngModelChange)="onPlaylistNameChange($event)"
12        placeholder="Új lista neve"
13      />
14      @if (playlistNameError()) {
15        <div class="playlist-error mb-2">{{ playlistNameError() }}</div>
16      }
17      <button class="btn create-btn w-100" [disabled]="!canCreate()" (click)="createPlaylist()">
18        + Lejátszási lista létrehozása
19      </button>
20    </div>
21  }
22  @if (statusMessage()) {
23    <div class="status-box mb-3">{{ statusMessage() }}</div>
24  }
25  <div class="list-group playlist-list">
26    @for (lista of playlists(); track lista.id) {
27      <button
28        type="button"
29        class="list-group-item list-group-item-action sidebar-link"
30        [class.active-playlist]="selectedPlaylistId() === lista.id"
31        (click)="selectPlaylist(lista.id)">
32        <div class="d-flex justify-content-between align-items-start gap-2">
33          <div>
34            <strong>{{ lista.nev }}</strong>
35            <div class="small text-light-emphasis">{{ lista.tetek.length }} db zene</div>
36          </div>
37          <div class="playlist-actions">
38            @if (selectedPlaylistId() === lista.id) {
39              <span class="badge text-bg-light">Aktív</span>
40            }
41            @if (user()) {
42              <button
43                type="button"
44                class="delete-x-btn"
45                title="Lejátszási lista törlése"
46                aria-label="Lejátszási lista törlése"
47                (click)="$event.stopPropagation(); deletePlaylist(lista.id)">
48                X
```

14. Kép: SidebarLeft

A bal oldali panel a felhasználó saját lejátszási listáinak kezelésére szolgál. Ez a komponens lehetővé teszi a lejátszási listák megjelenítését, létrehozását és törlését. A felhasználó új listát hozhat létre egy név megadásával, kiválaszthatja a meglévő lejátszási listái közül azt, amellyel dolgozni szeretne, illetve törölheti a már nem szükséges listákat. Emellett lehetőség van arra is, hogy egy playlistből eltávolítsunk egy adott dalt. Ez a komponens

szorosan együttműködik a központi állapotkezeléssel, így minden változás azonnal megjelenik a felületen.

Zenék kezelése (SidebarRight)

```
<div class="sidebar-right p-3 h-100 position-relative right-scroll">
  <h3 class="fw-bold mb-3">Zenék</h3>

  <div class="search-box mb-3">
    <input
      type="text"
      class="form-control"
      placeholder="Keresés dalcím szerint..."
      [ngModel]="searchTerm()"
      (ngModelChange)="searchTerm.set($event)"
    />
  </div>

  <div class="music-grid">
    @for (music of filteredMusics(); track music.id) {
      <div class="music-card" [class.selected]="playlistState.selectedSong()?.id === music.id">
        <div>
          <div class="music-title">{{ music.cim }}</div>
          <div class="music-meta">{{ music.eloado }}</div>
          <div class="music-meta">{{ music.hossza }}</div>
        </div>

        <button type="button" class="btn select-btn" (click)="selectSong(music)">
          | Kiválasztás
        </button>
        <button type="button" class="btn add-btn" (click)="addSelectedSong(music)">
          | Lejátszási listához adás
        </button>
      </div>
    } @empty {
      <div class="empty-search">Nincs találat erre a dalcímre.</div>
    }
  </div>
</div>
```

15. Kép: SidebarRight

A jobb oldali panel a zenék böngészésére szolgál. Itt jelenik meg az összes elérhető dal, amelyeket a felhasználó megtekinthet és kiválaszthat. A komponens lehetőséget biztosít keresésre is, így a felhasználó könnyen megtalálhatja a kívánt zenét. A keresés a frontend oldalon történik, tehát a már betöltött adatok szűrésével valósul meg. A kiválasztott dal hozzáadható az aktuálisan kiválasztott playlisthez, így a felhasználó könnyedén összeállíthatja saját zenei listáit.

Lejátszó (Player)

```

<div class="player-shell text-center my-4">
  <div class="record-wrapper">
    
  </div>

  <div class="current-song-label">{{ currentSongTitle() }}</div>

  <div class="player-controls">
    <button class="btn btn-dark control-btn" [disabled]="!canGoPrevious()" (click)="previousSong()">
      
    </button>

    <button class="btn btn-dark control-btn" [disabled]="!selectedSong()" (click)="togglePlay()">
      <img [src]="isPlaying() ? 'kepek/stop.png' : 'kepek/Play.png'" alt="Lejátszás" />
    </button>

    <button class="btn btn-dark control-btn" [disabled]="!canGoNext()" (click)="nextSong()">
      
    </button>
  </div>
</div>

```

16. Kép: Player

A Player komponens a kiválasztott dal megjelenítéséért és kezeléséért felel. Megjeleníti az aktuálisan kiválasztott dal címét, és lehetőséget biztosít a lejátszás vezérlésére. A felhasználó elindíthatja vagy megállíthatja a lejátszást, valamint léptethet az előző vagy következő dalra. A lejátszó jelenleg nem valódi audiolejátszást végez, hanem a kiválasztott dal állapotát kezeli, és a felhasználói felületen jeleníti meg az információkat.

Kommentek kezelése (Comments)

```

<div class="comments-shell mt-5">
  <div class="d-flex justify-content-between align-items-center mb-3 flex-wrap gap-2">
    <h3 class="m-0">Komentek</h3>
    <div class="selected-playlist-pill">
      {{ selectedPlaylist()?.nev ?? 'Nincs kiválasztott lejátszási lista' }}
    </div>
  </div>

  @if (selectedPlaylist()) {
    <div class="comment-form card mb-4">
      <div class="card-body">
        <textarea
          class="form-control mb-3"
          rows="3"
          placeholder="Írj kommentet az aktuális lejátszási listához..."
          [ngModel]="newComment()"
          (ngModelChange)="newComment.set($event)"
        ></textarea>
        <button class="btn btn-primary" [disabled]="!canSend()" (click)="addComment()">
          Komment küldése
        </button>
      </div>
    </div>

    @for (comment of comments(); track comment.id) {
      <div class="card my-3 comment-card">
        <div class="card-body">
          <div class="comment-header">
            <strong>{{ comment.felhasznalo?.felhasznalonev ?? ('Felhasználó #' + comment.felhasznalo_id) }}</strong>
            <span>{{ comment.letrehozva }}</span>
          </div>
          <p class="mb-0">{{ comment.uzenet }}</p>
        </div>
      </div>
    } @empty {
      <div class="empty-comments">Ehhez a lejátszási listához még nincs komment.</div>
    }
  } @else {
    <div class="empty-comments">Válassz ki egy lejátszási listát, és itt megjelennek a kommentek.</div>
  }
</div>

```

17. Kép: Comments komponens

A Comments komponens a playlistekhez tartozó kommentek megjelenítésére és kezelésére szolgál. Megjeleníti az aktuálisan kiválasztott playlisthez tartozó hozzászólásokat, és lehetőséget biztosít új komment írására. Komment hozzáadására csak akkor van lehetőség, ha a felhasználó be van jelentkezve, és ki van választva egy playlist. Ez biztosítja, hogy a kommentek mindig egy adott playlisthez kapcsolódjanak.

Bejelentkezés és regisztráció (Auth)

```

@if (isLoginMode()) {
  <form [formGroup]="loginForm" (ngSubmit)="submitLogin()">
    <label>Email</label>
    <input type="email" formControlName="email" placeholder="Email cím" />
    @if (loginForm.controls.email.touched && loginForm.controls.email.hasError('required')) {
      <div class="field-error">Az email cím megadása kötelező.</div>
    }
    @if (loginForm.controls.email.touched && loginForm.controls.email.hasError('email')) {
      <div class="field-error">Adj meg egy valós email címet.</div>
    }

    <label>Jelszó</label>
    <input type="password" formControlName="password" placeholder="Jelszó" />
    @if (loginForm.controls.password.touched && loginForm.controls.password.hasError('required')) {
      <div class="field-error">A jelszó megadása kötelező.</div>
    }
    @if (loginForm.controls.password.touched && loginForm.controls.password.hasError('minlength')) {
      <div class="field-error">A jelszónak legalább 6 karakterből kell állnia.</div>
    }

    <button type="submit" [disabled]="loading()">{{ loading() ? 'Beléptetés...' : 'Bejelentkezés' }}</button>
  </form>
}

```

18. Kép: Bejelentkezési űrlap és validáció

Az Auth komponens a felhasználók hitelesítéséért felel, vagyis itt történik a bejelentkezés és a regisztráció kezelése. A komponens Angular Reactive Forms megoldást használ, amely lehetővé teszi az űrlap mezők egyszerű kezelését és validálását. A képen látható, hogy a bejelentkezési űrlap két fő mezőt tartalmaz: email és jelszó. Ezek a mezők a formGroup segítségével vannak összekapcsolva a TypeScript oldalon definiált formmal. Az ngSubmit esemény kezeli az űrlap elküldését, amely meghívja a megfelelő függvényt.


```

@else {
<form [formGroup]="registerForm" (ngSubmit)="submitRegister()">
  <label>Felhasználónév</label>
  <input type="text" formControlName="felhasznalonev" placeholder="Felhasználónév" />
  @if (registerForm.controls.felhasznalonev.touched && registerForm.controls.felhasznalonev.hasError('required')) {
    <div class="field-error">A felhasználónév megadása kötelező.</div>
  }
  @if (registerForm.controls.felhasznalonev.touched && registerForm.controls.felhasznalonev.hasError('minlength')) {
    <div class="field-error">A felhasználónév legalább 3 karakter legyen.</div>
  }

  <label>Email</label>
  <input type="email" formControlName="email" placeholder="Email cím" />
  @if (registerForm.controls.email.touched && registerForm.controls.email.hasError('required')) {
    <div class="field-error">Az email cím megadása kötelező.</div>
  }
  @if (registerForm.controls.email.touched && registerForm.controls.email.hasError('email')) {
    <div class="field-error">Adj meg egy érvényes email címet.</div>
  }

  <label>Jelszó</label>
  <input type="password" formControlName="password" placeholder="Jelszó" />
  @if (registerForm.controls.password.touched && registerForm.controls.password.hasError('required')) {
    <div class="field-error">A jelszó megadása kötelező.</div>
  }
  @if (registerForm.controls.password.touched && registerForm.controls.password.hasError('minlength')) {
    <div class="field-error">A jelszónak legalább 6 karakter hosszúnak kell lennie.</div>
  }

  <label>Jelszó újra</label>
  <input type="password" formControlName="confirmPassword" placeholder="Jelszó újra" />
  @if (registerForm.controls.confirmPassword.touched && registerForm.controls.confirmPassword.hasError('required')) {
    <div class="field-error">Írd be újra a jelszót.</div>
  }
  @if (registerPasswordMismatch()) {
    <div class="field-error">A két jelszó nem egyezik meg.</div>
  }

  <button type="submit" [disabled]="loading()">{{ loading() ? 'Mentés...' : 'Regisztráció' }}</button>
</form>

```

19. Kép: Validáció működése

A validáció fontos része az űrlap működésének. Az Angular ellenőrzi, hogy az email mező ki van-e töltve, illetve hogy megfelelő formátumú-e. A jelszó esetében szintén kötelező a kitöltés, és minimum hossz is meg van adva (6 karakter). Ha a felhasználó hibás adatot ad meg, akkor a rendszer azonnal hibaüzenetet jelenít meg a mező alatt. A hibakezelés feltételes megjelenítéssel történik, például csak akkor jelenik meg a hibaüzenet, ha a mezőt már módosította a felhasználó, és az nem felel meg a feltételeknek. Ez javítja a felhasználói élményt, mert nem jelennek meg feleslegesen hibák már az oldal betöltésekor. A bejelentkezés gomb dinamikusan változik az állapot függvényében. Ha folyamatban van a belépés, akkor a gomb szövege „Beléptetés...” lesz, és a gomb letiltásra kerül. Ez megakadályozza, hogy a felhasználó többször egymás után elküldje az űrlapot.

Műsorok oldal

```

<section class="shows-page">
  <div class="container py-5">
    <div class="section-head mb-4">
      <p class="eyebrow">Műsorok</p>
      <h1>Heti toplista</h1>
      <p>
        Ezeket a műsorokat hallhatod nálunk a héten. Minden napra egy új zenei utazás, tele izgalmas történetekkel és fantasztikus zenékkal.
      </p>
    </div>

    <div class="row g-4" *ngIf="hetiMusorok$ | async as hetiMusorok">
      <div class="col-12 col-xl-6" *ngFor="let napiMusor of hetiMusorok">
        <article class="show-card h-100">
          <div class="card-header-row">
            <div>
              <span class="day-pill">{{ napiMusor.nap }}</span>
              <h2>{{ napiMusor.cim }}</h2>
            </div>
            <span class="host-name">{{ napiMusor.musorvezeto }}</span>
          </div>

          <p class="description">{{ napiMusor.leiras }}</p>

          <h3>Lejátszott zenék</h3>
          <ul class="song-list">
            <li *ngFor="let dal of napiMusor.dalok">
              <strong>{{ dal.eloado }}</strong> - {{ dal.cim }}
              <span>{{ dal.hossza }}</span>
            </li>
          </ul>
        </article>
      </div>
    </div>
  </div>
</section>

```

20. Kép: Műsorok oldal

A műsorok oldal egy összetettebb logikát tartalmazó rész, amely több különböző adatforrást kombinál. Az oldal egy heti műsortervet állít össze a rendelkezésre álló adatok alapján. Ez a komponens nem csupán megjeleníti az adatokat, hanem feldolgozza és rendszerezi is azokat, így dinamikus generált tartalmat jelenít meg.

Műsorvezetők oldal

```

<section class="listing-page">
  <div class="container py-5">
    <div class="section-head mb-4">
      <p class="eyebrow">Műsorvezetők</p>
      <h1>Akik hangot adnak a kedvenc pillanataidnak.</h1>
      <p>Fedezd fel műsorvezetőinket, és ismerd meg, kik szólnak hozzád a rádió hullámhosszán.</p>
    </div>

    <div class="row g-4" *ngIf="musorvezetok$ | async as musorvezetok">
      <div class="col-12 col-md-6 col-xl-4" *ngFor="let musorvezeto of musorvezetok">
        <article class="info-card h-100">
          <div class="card-top">
            <div class="avatar">{{ musorvezeto.nev.charAt(0) }}</div>
            <span class="tag">ID #{{ musorvezeto.id }}</span>
          </div>
          <h2>{{ musorvezeto.nev }}</h2>
          <p>{{ musorvezeto.bemutakozas }}</p>
        </article>
      </div>
    </div>
  </div>
</section>

```

21. Kép: Műsorvezetők oldal

A műsorvezetők oldal a rendszerben szereplő műsorvezetők megjelenítésére szolgál. Az oldal célja, hogy a felhasználó áttekinthető formában lássa, kik tartoznak az alkalmazáshoz, és rövid leírást kapjon róluk. A felület felső részén egy cím és egy rövid bemutató szöveg található, amely leírja az oldal tartalmát. Ez alatt jelenik meg a műsorvezetők listája. Az adatok betöltése aszinkron módon történik, és az Angular *ngIf valamint async pipe segítségével kerülnek megjelenítésre. Ez biztosítja, hogy az oldal csak akkor jelenítse meg a tartalmat, amikor az adatok már rendelkezésre állnak. A lista megjelenítése egy *ngFor ciklussal történik, amely végigmegy a műsorvezetők tömbjén, és minden elemhez egy külön kártyát hoz létre. Ezek a kártyák tartalmazzák a műsorvezető nevét, azonosítóját és bemutatkozását. A megjelenítés reszponzív rácsszerkezetben történik, így az oldal különböző képernyőméretekken is jól használható marad.

Állapotkezelés – AuthStateService

```
import { Injectable, computed, signal } from '@angular/core';
import { User } from '../models/user.model';

@Injectable({ providedIn: 'root' })
export class AuthStateService {
  private readonly storageKey = 'szikszii_user';
  readonly user = signal<User | null>(this.readStoredUser());
  readonly isLoggedIn = computed(() => this.user() !== null);
  readonly displayName = computed(() => this.user()?.felhasznalonev ?? 'Bejelentkezés');

  setUser(user: User): void {
    this.user.set(user);
    localStorage.setItem(this.storageKey, JSON.stringify(user));
  }

  logout(): void {
    this.user.set(null);
    localStorage.removeItem(this.storageKey);
  }

  private readStoredUser(): User | null {
    const raw = localStorage.getItem(this.storageKey);
    if (!raw) {
      return null;
    }

    try {
      return JSON.parse(raw) as User;
    } catch {
      localStorage.removeItem(this.storageKey);
      return null;
    }
  }
}
```

22. Kép: AuthStateService

Ez a service a felhasználói állapot kezeléséért felel. Tárolja az aktuális felhasználót, és meghatározza, hogy a felhasználó be van-e jelentkezve. A service gondoskodik arról is, hogy a felhasználó adatai megmaradjanak az oldal újratöltése után is, például a localStorage használatával. Emellett kezeli a kijelentkezést is.

PlaylistStateService

```
createPlaylist(userId: number, nev: string): Observable<{ success: boolean; message: string }> {
  return this.http.post<{ success: boolean; message: string }>(`${this.apiUrl}/create-playlist`, {
    felhasznalo_id: userId,
    nev,
  }).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}

addSongToPlaylist(playlistId: number, dalId: number, userId: number): Observable<{ success: boolean; message: string }> {
  return this.http.post<{ success: boolean; message: string }>(`${this.apiUrl}/${playlistId}/songs`, {
    dal_id: dalId,
  }).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}

removeSongFromPlaylist(playlistId: number, tetelId: number, userId: number): Observable<{ success: boolean; message: string }> {
  return this.http.delete<{ success: boolean; message: string }>(`${this.apiUrl}/${playlistId}/songs/${tetelId}`).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}

deletePlaylist(playlistId: number, userId: number): Observable<{ success: boolean; message: string }> {
  return this.http.delete<{ success: boolean; message: string }>(`${this.apiUrl}/${playlistId}/delete-playlist`).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}
```

23. Kép: PlaylistStateService

Ez a frontend egyik legfontosabb eleme, amely a playlistekhez kapcsolódó adatokat és állapotokat kezeli. Tárolja a playlisteket, a kiválasztott playlistet, a kiválasztott dalt, valamint a kommenteket. Emellett biztosítja a dalok közötti navigációt és a különböző műveletek végrehajtását. Ez a service biztosítja, hogy a felhasználói felület mindig szinkronban legyen az aktuális állapottal.

Service-ek szerepe

```

TS auth-state.service.ts
TS comment-service.spec.ts
TS comment-service.ts
TS music-service.spec.ts
TS music-service.ts
TS musor-service.ts
TS musorvezeto-service.ts
TS playlist-service.spec.ts
TS playlist-service.ts
TS playlist-state.service.ts

```

24. Kép: Service-ek

A service-ek az adatkezelésért és a backenddel való kommunikációért felelnek. Ezek biztosítják, hogy a frontend hozzáférjen a szükséges adatokhoz. A különböző service-ek különböző típusú adatokat kezelnek, például felhasználók, zenék, playlistek, vagy kommentek. Ez a réteg biztosítja az alkalmazás működésének hátterét, és elkülöníti a logikát a megjelenítéstől.

Routing

```

export const routes: Routes = [
  { path: '', component: Home },
  { path: 'rolunk', component: About },
  { path: 'musorvezetok', component: MusorvezetokPage },
  { path: 'musorok', component: MusorokPage },
  { path: 'auth', component: Auth },
  { path: '**', redirectTo: '' },
];

```

25. Kép: Routing

A frontend alkalmazásban az oldalak közötti navigációt az Angular Router kezeli. A routing lényege, hogy az alkalmazás az URL alapján dönti el, melyik komponens jelenjen meg a képernyőn. Ennek köszönhetően a felhasználó különböző oldalak között tud váltani úgy, hogy közben az alkalmazás nem töltődik újra teljesen, csak a megfelelő tartalom cserélődik ki. A csatolt kódrészletben a route-ok egy tömbben vannak definiálva, ahol minden útvonalhoz hozzá van rendelve egy komponens. Ez a megoldás átláthatóvá teszi a navigációs logikát, mert egy helyen jól látható, hogy melyik URL melyik oldalhoz tartozik.

A path: '' beállítás a kezdőoldalt jelenti. Ez azt mondja meg, hogy ha a felhasználó az alkalmazás gyökércímére lép, akkor a Home komponens töltődjön be. Ez gyakorlatilag az alkalmazás alapértelmezett nyitóoldala, ahová a felhasználó először érkezik.

A path: 'rolunk' útvonal az About komponenshez tartozik. Ez az oldal általában a projektről vagy az oldal céljáról ad információt, tehát inkább statikus tartalmat jelenít meg. A routing itt azt biztosítja, hogy a „Rólunk” menüpontra kattintva a megfelelő komponens jelenjen meg.

A path: 'musorvezetok' útvonal a MusorvezetokPage komponenst tölti be. Ez az oldal a műsorvezetők megjelenítésére szolgál, vagyis külön route-ot kapott, hogy jól elkülönüljön a többi tartalomtól. Ugyanez a logika működik a path: 'musorok' esetében is, ahol a MusorokPage komponens jelenik meg, tehát a műsorok külön saját oldalt kaptak.

A path: 'auth' útvonal az Auth komponenshez tartozik. Ez a route a bejelentkezési és regisztrációs funkciókat kezeli. Külön útvonalon szerepel, mert ez az alkalmazás egyik fontos, jól elkülöníthető funkcionális része. Így a felhasználó közvetlenül is elérheti a hitelesítéshez tartozó oldalt.

A kódban szerepel egy path: '**' beállítás is, amely az úgynevezett wildcard route. Ennek az a szerepe, hogy elkapjon minden olyan URL-t, ami a korábban definiált útvonalak egyikéhez sem tartozik. Ilyenkor az alkalmazás nem hibára fut, hanem visszairányítja a felhasználót a kezdőoldalra a redirectTo: '' beállítás segítségével. Ez felhasználói szempontból hasznos, mert elkerülhető vele, hogy egy hibás vagy nem létező cím miatt üres vagy rossz oldal jelenjen meg.

Tesztelés

A fejlesztés során kiemelt figyelmet kapott az alkalmazás megbízhatóságának biztosítása, amelyet egységtesztok és végfelhasználói (E2E) tesztek alkalmazásával valósítottunk meg. A tesztelés célja az egyes funkciók helyes működésének ellenőrzése, valamint a hibák korai felismerése volt. Az egységtesztok megvalósításához a Vitest tesztkeretrendszer került alkalmazásra. A Vitest egy modern, nagy teljesítményű tesztelő eszköz, amely gyors futási időt és egyszerű konfigurációt biztosít. A tesztek futtatása az ng test paranccsal történik. A futtatás során az eredmények közvetlenül a terminálban jelennek meg, így nincs szükség külön grafikus felületre. A tesztfutató folyamatos figyelési módban működik, tehát a fájlok módosítása esetén automatikusan újrafuttatja a teszteket. Az alkalmazás főbb egységei külön-külön tesztelésre kerültek:

Komponensek:

- Footer komponens
- Navbar komponens
- Home oldal
- Player komponens
- Comments komponens
- Sidebar (bal és jobb oldali)

Szolgáltatások:

- PlaylistService
- MusicService
- CommentService

A tesztek ellenőrzik például: a komponensek létrejöttét, a szolgáltatások működését, az alapvető logikai működést. A tesztek futtatása során 12 tesztből 12 sikeresen lefutott. A tesztek gyorsan és hibamentesen lefutottak, ami azt mutatja, hogy az alkalmazás komponensei stabilan működnek.

Tesztfuttatás eredménye:

```
DEV v4.0.18 C:/Users/User/Desktop/FRONTEND/FRONTEND
✓ SziksziFM src/app/footer/footer.spec.ts (1 test) 166ms
✓ SziksziFM src/app/playlist-service.spec.ts (1 test) 7ms
✓ SziksziFM src/app/player/player.spec.ts (1 test) 196ms
✓ SziksziFM src/app/music-service.spec.ts (1 test) 4ms
✓ SziksziFM src/app/comment-service.spec.ts (1 test) 6ms
✓ SziksziFM src/app/comments/comments.spec.ts (1 test) 185ms
✓ SziksziFM src/app/sidebar-left/sidebar-left.spec.ts (1 test) 232ms
✓ SziksziFM src/app/sidebar-right/sidebar-right.spec.ts (1 test) 242ms
✓ SziksziFM src/app/navbar/navbar.spec.ts (1 test) 205ms
✓ SziksziFM src/app/home/home.spec.ts (1 test) 324ms
✓ should create 322ms
✓ SziksziFM src/app/app.spec.ts (2 tests) 266ms

Test Files 11 passed (11)
Tests 12 passed (12)
Start at 18:50:44
Duration 3.16s (transform 882ms, setup 7.21s, import 2.02s, tests 1.83s, environment 14.55s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

26. Kép: Tesztfuttatás eredménye

Az alábbi ábra a tesztfuttatás eredményét mutatja a terminálban, ahol látható, hogy minden teszt sikeresen lefutott.

Az alkalmazás végfelhasználói működésének ellenőrzésére Playwright alapú end-to-end (E2E) tesztelés került alkalmazásra. A tesztek futtatása az `ng e2e` paranccsal történik. A Playwright lehetővé teszi az alkalmazás több böngészőbe történő tesztelését: Chromium, Firefox, WebKit.

E2E teszteredmények:

Lefuttatott tesztek száma: 3

Sikeres tesztek: 3

A tesztek minden böngészőben sikeresen lefutottak, ami igazolja az alkalmazás kompatibilitását és stabil működését.

A Vitest és a Playwright alkalmazásával egy modern és hatékony tesztelési környezet került kialakításra. Az egységtesztek és E2E tesztek együttesen biztosítják az alkalmazás megbízható működését, valamint támogatják a további fejlesztést és karbantartást.

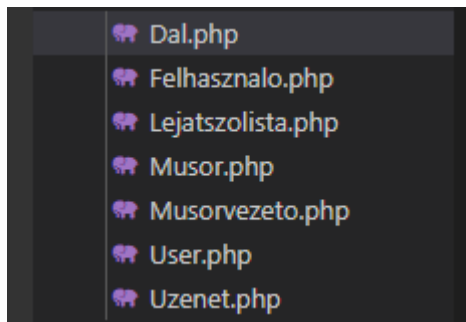
Backend

Alkalmazás backend részének részletes leírása + képernyőfotók

Bevezetés

Ez a dokumentáció egy webalkalmazás backend részének elkészítését mutatja be, a tervezéstől egészen a megvalósításig. Bemutatjuk, hogyan áll össze a háttérben az a rendszer, ami kiszolgálja a frontend kéréseit, kezeli az adatokat, és életben tartja az alkalmazást.

Modellek:



27. Kép: Modellek

Az adatbázisból kifolyólag 7 db Modellre volt szükségünk

Mikre jók ezek az osztályok?

- A Dal osztály a dalok adatbázistáblához tartozó modell. Ez az osztály teszi lehetővé, hogy a táblában lévő adatokat objektumorientált módon érjük el, anélkül, hogy nyers SQL lekérdezéseket kellene írni.

```
class Dal extends Model
{
    use HasFactory;
    public $table='dalok';
    public $timestamps=false;
    public $guarded=[];
}
```

28. Kép: Modell példa

Mit csinálnak az egyes részek?

`'class Dal extends Model'`

- Ez jelzi, hogy a Dal egy modell, vagyis egy adatbázistábla logikai reprezentációja.

`'use HasFactory;'`

- Lehetővé teszi, hogy factorykat használjunk a modellhez, például teszteléshez vagy mintaadatok generálásához.

```
'public $table = 'dalok';'
```

- Alapértelmezés szerint a Laravel a modell nevét többes számban használná (dals), ezért itt explicit módon megadjuk, hogy a modell a dalok nevű táblához tartozik.

```
'public $timestamps = false;'
```

- Kikapcsolja az automatikus created_at és updated_at mezők kezelését. Erre akkor van szükség, ha az adatbázistábla nem tartalmaz ilyen oszlopokat.

```
'public $guarded = [];'
```

- Ez azt jelenti, hogy nincs védett mező, vagyis minden oszlop tömegesen feltölthető (mass assignment). Ez egyszerűsíti az adatmentést, de csak akkor biztonságos, ha a bemeneti adatokat már megfelelően validáltuk.

```
public function dal()  
{  
    return $this->belongsTo(Dal::class, 'dal_id');  
}
```

29. kép: BelongsTo függvény

- Néhol szükségünk volt belongsTo függvényt is használni.

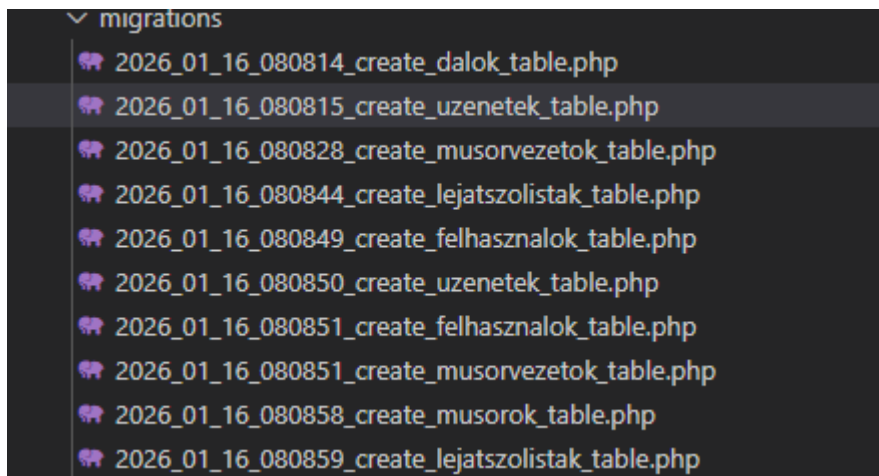
Mit csinál ez a függvény?

A dal() egy modellfüggvény, amely egy kapcsolatot definiál az aktuális modell és a Dal modell között. A függvény meghívásakor megtudja, hogy az adott rekord egy dalhoz tartozik, amit a dal_id idegen kulcs azonosít.

Hogyan működik?

- A függvény visszatérési értéke egy belongsTo kapcsolat
- Ez nem egy „sima” üzleti logikát tartalmazó függvény, hanem egy kapcsolatdefiníció

Migráció:



30. Kép: Migrációk

Az adatbázisunk minden adattáblájához csinálnunk kellett egy migrációt.

```
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('uzenetek', function (Blueprint $table) {
            $table->id();
            $table->foreignId('felhasznalo_id')->references('id')->on('felhasznalok');
            $table->string('uzenet');
            $table->date('letrehozva');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('uzenetek');
    }
};
```

31. Kép: Migráció példa

Mire szolgál ez a migráció?

Ez a migráció az `uzenetek` nevű adatbázistáblát hozza létre. A tábla célja, hogy eltárolja a felhasználók által létrehozott üzeneteket, és összekapcsolja őket a megfelelő felhasználóval.

Mit csinál az `up()` függvény?

Az `up()` függvény határozza meg, hogy milyen változtatások történjenek az adatbázisban a migráció futtatásakor.

Létrehoz egy új táblát `uzenetek` néven.

Egy automatikusan növekvő elsődleges kulcs (`id`) mezőt hoz létre.

Egy idegen kulcsot definiál, amely a `felhasznalok` tábla `id` mezőjére hivatkozik. Ez biztosítja, hogy minden üzenet egy létező felhasználóhoz tartozzon.

Az üzenet szövegét tároló mező, szöveges (`VARCHAR`) formátumban.

Az `uzenet` létrehozásának dátumát tárolja. Ebben az esetben nem a Laravel automatikus `created_at` mezőjét használja, hanem egy saját, kézzel kezelt dátummezőt.

Mit csinál a `down()` függvény?

Ez a függvény a migráció visszavonásáért felel. Ha a migrációt rollbackeljük, az `uzenetek` tábla törlésre kerül az adatbázisból.

Miért hasznos ez backend szempontból?

Ez a migráció:

- egységesen létrehozza az üzenetek adatbázis-struktúráját,

- biztosítja az adatok közti kapcsolatot (felhasználó ↔ üzenet),
- lehetővé teszi az egyszerű telepítést és visszavonást,
- segít elkerülni a kézi adatbázis-módosításokból eredő hibákat.

Sedeer:

Nekünk az adatbázis adatainak feltöltése minden tábla esetén migrációval történik

```
$dalok=
[
    [1, 'Queen', 'Bohemian Rhapsody', '00:05:55'],
    [2, 'Metallica', 'Nothing Else Matters', '00:06:28'],
    [3, 'Coldplay', 'Fix You', '00:04:55'],
    [4, 'Adele', 'Someone Like You', '00:04:45'],
    [5, 'Daft Punk', 'Get Lucky', '00:06:07'],
    [6, 'Pink Floyd', 'Wish You Were Here', '00:05:34'],
    [7, 'Nirvana', 'Come As You Are', '00:03:39'],
    [8, 'The Beatles', 'Let It Be', '00:04:03'],
    [9, 'U2', 'Beautiful Day', '00:04:08'],
    [10, 'Red Hot Chili Peppers', 'Under The Bridge', '00:04:24'],
    [11, 'Imagine Dragons', 'Radioactive', '00:03:06'],
    [12, 'AC/DC', 'Highway to Hell', '00:03:28'],
    [13, 'Eminem', 'Stan', '00:06:44'],
    [14, 'Madonna', 'Frozen', '00:06:12'],
    [15, 'Linkin Park', 'In The End', '00:03:36'],
    [16, 'Hans Zimmer', 'Time', '00:04:35'],
    [17, 'Lana Del Rey', 'Video Games', '00:04:42'],
    [18, 'Depeche Mode', 'Enjoy the Silence', '00:04:15'],
    [19, 'Radiohead', 'No Surprises', '00:03:49'],
    [20, 'Bon Jovi', 'Always', '00:05:53']
];
foreach($dalok as $key=>$value)
{
    Dal::create([
        'id'=>$value[0],
        'eloado'=>$value[1],
        'cim'=>$value[2],
        'hossza'=>$value[3]
    ]);
}
```

32. Kép: Seeder példa

Mire jó ez a kód?

Ez a kód előre definiált daladatokat tölt fel az adatbázisba a Dal modellen keresztül. Tipikusan seedelésre (teszt- vagy mintaadatok feltöltésére) használjuk.

Mit csinálnak az egyes részek?

```
$dalok = [ ... ];
```

Egy tömb, ami a beszúrni kívánt dalokat tartalmazza (ID, előadó, cím, hossz).

```
foreach ($dalok as $key => $value)
```

Végigiterál a dalokon egyesével.

```
Dal::create([
```

Az create() metódusával új rekordot szúr be a dalok táblába.

```
'id' => $value[0],  
'eloado' => $value[1],  
'cim' => $value[2],  
'hossza' => $value[3]
```

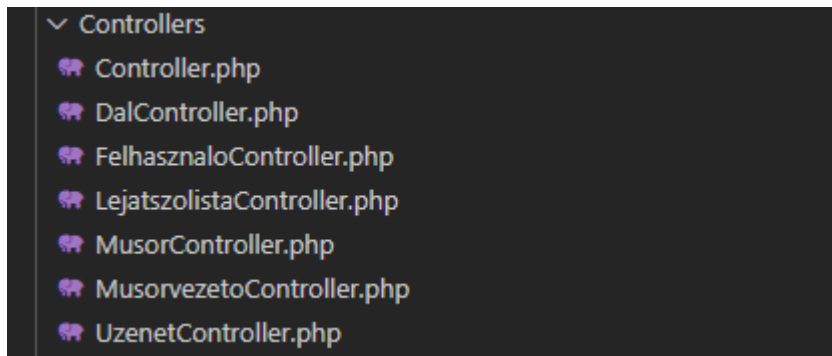
Az aktuális dal adatait rendeli hozzá az adatbázis mezőjéhez.

Összefoglalva

Ez a kód:

- mintaadatokat visz fel az adatbázisba,
- gyors tesztelést tesz lehetővé,
- és biztosítja, hogy a backend induláskor tartalmazzon alap adatokat.

Controllerek:



33. Kép: Controllerek

Mikre jók a Controllerek?

Ez a kontroller a lejátszólisták backend oldali kezeléséért felel.

Az API-n keresztül biztosítja a CRUD műveleteket (lekérdezés, létrehozás, módosítás, törlés) a Lejatszolista modellhez tartozó adatokon.

Mit csinálnak egyes részei??

index():

- Lekéri az **összes lejátszólista rekordot**
- A kapcsolódó dalokat is betölti (`with('dal')`)
- Tömeges lekérdezéshez használható

```
public function index()
{
    return Lejatszolista::with('dal')->get();
}
```

34. Kép: Controller get függvény

getById(\$id):

- Egy lejátszólistát kér le azonosító alapján
- Ha nem létezik, 404-es hibát ad vissza
- Egy konkrét elem megjelenítésére szolgál

```

public function getById($id)
{
    $lejatszoLista=LejatszoLista::find($id);
    if(is_null($lejatszoLista))
    {
        return response()->json(['Azonosító hiba:'=>'Nincs ilyen id-jű lejátsszólista.'],404);
    }
    return response()->json($lejatszoLista,208);
}

```

35. Kép: Controller getById függvény

store(Request \$request):

- Új lejátsszólista létrehozása
- Ellenőrzi a kötelező mezők meglétét
- Sikeres mentés esetén új rekordot hoz létre az adatbázisban

```

public function store(Request $request)
{
    $validator=Validator::make($request->all(),
    [
        'dal_id'=>'required',
        'sorrend_szam'=>'required',
        'musor_id'=>'required'
    ]);
    if($validator->fails())
    {
        return response()->json(['Hiba:'=>'Fontos adat hiányzik.'],406);
    }
    $lejatszoLista=LejatszoLista::create($request->all());
    return response()->json(['Új lejátsszólista létrehozva, a következő ID-vel:'=>$lejatszoLista->id],201);
}

```

36. Kép: Controller store függvény

update(Request \$request, \$id):

- Meglévő lejátsszólista módosítása
- Ellenőrzi, hogy létezik-e a megadott ID
- Validálja a beérkező adatokat
- Frissíti az adatbázisban lévő rekordot


```

public function update(Request $request, $id)
{
    $lejatszalista=Lejatszalista::find($id);
    if(is_null($lejatszalista))
    {
        return response()->json(['Hiba:'=>'Nem található a lejátszólista.'],404);
    }
    $validator=Validator::make($request->all(),
    [
        'dal_id'=>'required',
        'sorrend_szam'=>'required',
        'musor_id'=>'required'
    ]);
    if($validator->fails())
    {
        return response()->json(['Hiba:'=>'Fontos adat hiányzik.'],401);
    }
    $lejatszalista->update($request->all());
    return response()->json(['Az alábbi lejátszólista adatai módosultak:'=>$lejatszalista->id],201);
}

```

37. Kép: Controller update függvény

destroy(\$id):

- Lejátszólista törlése azonosító alapján
- Hibaüzenetet ad, ha nem található
- Sikeres törlés esetén eltávolítja az adatot az adatbázisból

```

public function destroy($id)
{
    $lejatszalista=Lejatszalista::find($id);
    if(is_null($lejatszalista))
    {
        return response()->json(['Hiba:'=>'Nem található az adott ID-jű lejátszólista.'],404);
    }
    $lejatszalista->delete();
    return response('Lejátszólista sikeresen törölve.',202);
}

```

38. Kép: Controller destroy függvény

getOlderShow(\$letrehozas):

- Egy megadott dátumnál **régebbi műsorokat** kér le
- Az adatbázisban a letrehozva mező alapján szűr (< \$letrehozas)
- Ellenőrzi, hogy létezik-e találat
- Találat esetén visszaadja a műsorok listáját

- Ha nincs találat, **404-es hibát** ad vissza megfelelő hibaüzenettel

```

}
public function getOlderShow($letrehozás)
{
    $musor=Musor::where('letrehozva','<',$letrehozás);
    if($musor->exists())
    {
        return $musor->get();
    }
    else
    {
        return response()->json(['Hiba:'=>'Nem található a megadott értéknél régebbi műsor.'],404);
    }
}

```

39. Kép: Controller getOlder függvény

getNewerShow(\$letrehozás):

- Egy megadott dátumnál **újabb műsorokat** kér le
- Az adatbázisban a letrehozva mező alapján szűr (> \$letrehozás)
- Ellenőrzi, hogy létezik-e találat
- Találat esetén visszaadja a műsorok listáját
- Ha nincs találat, **404-es hibát** ad vissza megfelelő hibaüzenettel

```

}
public function getNewerShow($letrehozás)
{
    $musor=Musor::where('letrehozva','>',$letrehozás);
    if($musor->exists())
    {
        return $musor->get();
    }
    else
    {
        return response()->json(['Hiba:'=>'Nem található a megadott értéknél újabb műsor.'],404);
    }
}

```

40. Kép: Controller getNewer függvény

Összegzés:

Ez a kontroller metódusai:

- az API végpontok logikáját valósítják meg,
- biztosítják a lejártszólisták CRUD műveleteit,
- és kezelik a hibás kéréseket is.

Api végpontok:

Ezek Laravel keretrendszerben írt API-útvonalak (route-ok), amelyek a backendben meghatározzák, hogy mely HTTP-kérésekhez melyik vezérlő (controller) metódus tartozik.

Példa:

```
Route::get('/felhasznalok',[FelhasznaloController::class,'index']);
Route::get('/felhasznalok/{id}',[FelhasznaloController::class,'getId'])->whereNumber('id');
Route::post('/felhasznalok',[FelhasznaloController::class,'store']);
Route::put('/felhasznalok/{id}',[FelhasznaloController::class,'update']);
Route::get('/felhasznalok/filterby/{szerep}',[FelhasznaloController::class,'filterByRole']);
Route::get('/felhasznalok/searchbyname',[FelhasznaloController::class,'searchByName']);
Route::delete('/felhasznalok/{id}',[FelhasznaloController::class,'destroy']);
```

41. Kép: Api végpontok

Röviden: mind a felhasználók kezelésére szolgálnak — tipikus CRUD műveletek (Create, Read, Update, Delete).

Route::get('/felhasznalok', 'index')

→ Az összes felhasználót lekéri.

Route::get('/felhasznalok/{id}', 'getId')

→ Egy konkrét felhasználót ad vissza az id alapján.

Route::post('/felhasznalok', 'store')

→ Új felhasználót hoz létre .

Route::put('/felhasznalok/{id}', 'update')

→ Egy meglévő felhasználó adatait frissít.

Route::get('/felhasznalok/filterby/{szerep}', 'filterByRole')

→ Szerep (role) alapján szűri a felhasználókat.

Route::get('/felhasznalok/searchbyname', 'searchByName')

→ Név alapján keres a felhasználók között.

Route::delete('/felhasznalok/{id}', 'destroy')

→ Törli a megadott felhasználót .

Összefoglalva:

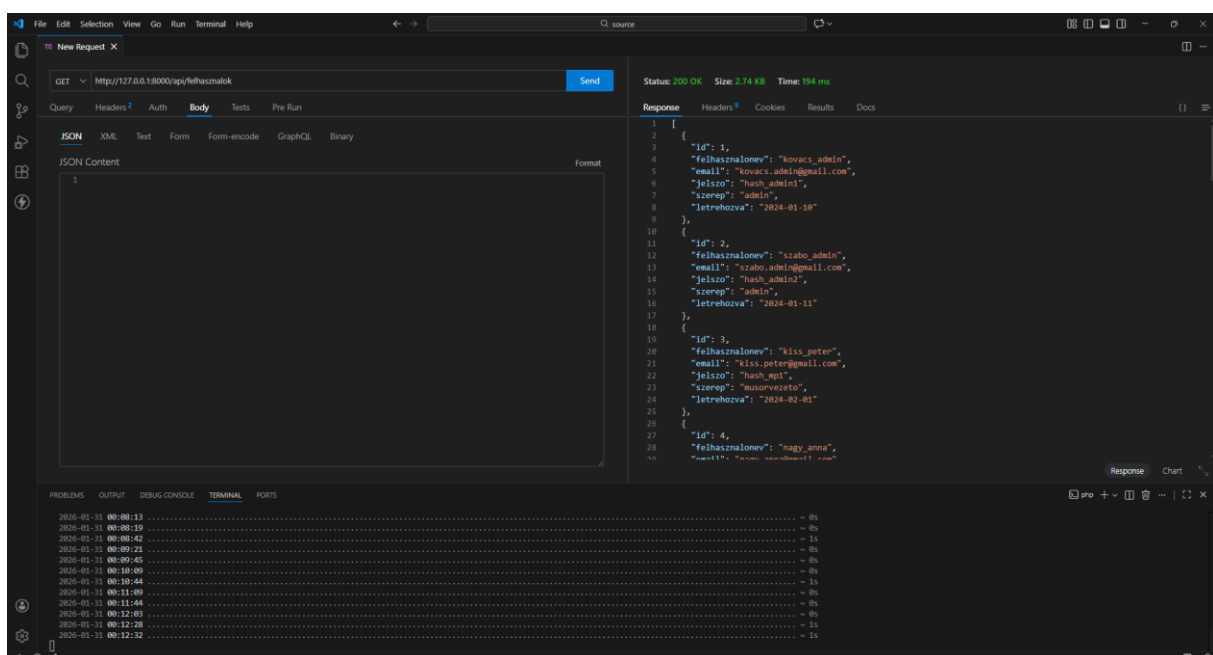
Ezek az útvonalak határozzák meg, hogyan kezeli a backend a felhasználókkal kapcsolatos kéréseket (pl. listázás, hozzáadás, módosítás, törlés).

Tesztek:

Miután végeztünk a Backend egészének megírásával, elkezdődtek a tesztek. A tesztek közben azt vizsgáltuk hogy az egyes lekérdezések a megfelelő értéket adják-e vissza, a megfelelő formátumban. Erre a Thunder Client nevezetű VS code-os bővítményt használtuk amivel a backend api végpontokat tudtuk tesztelni.

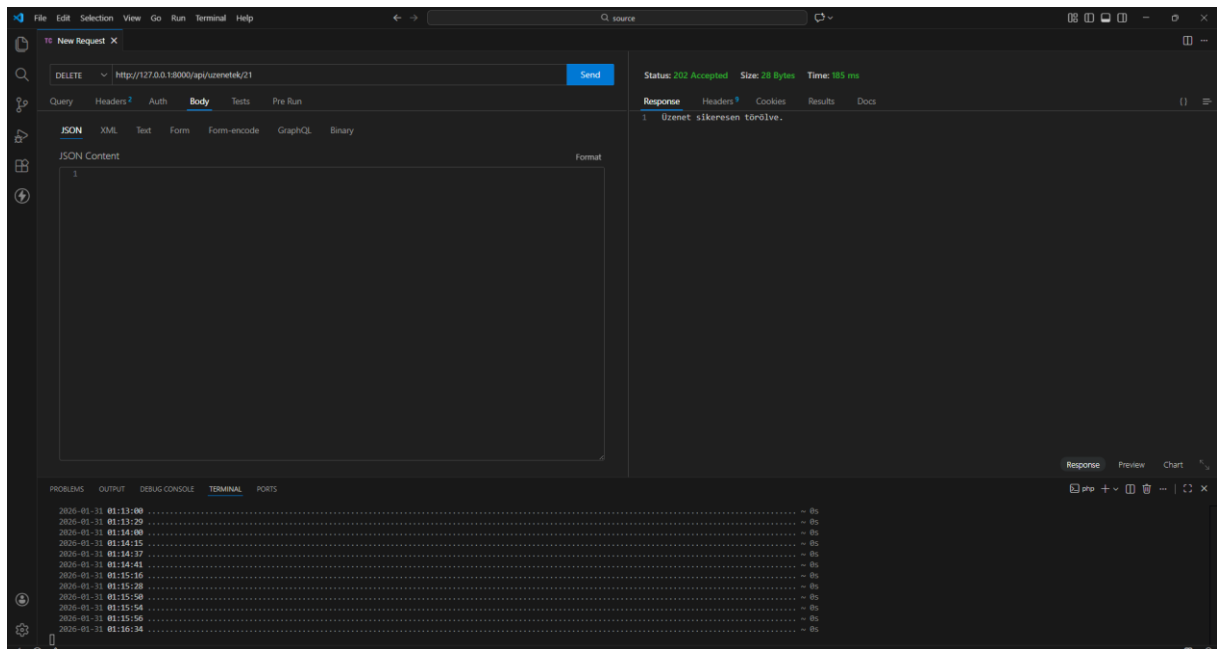
Erre pár példa:

Felhasználók get metódus sikeres teszt:



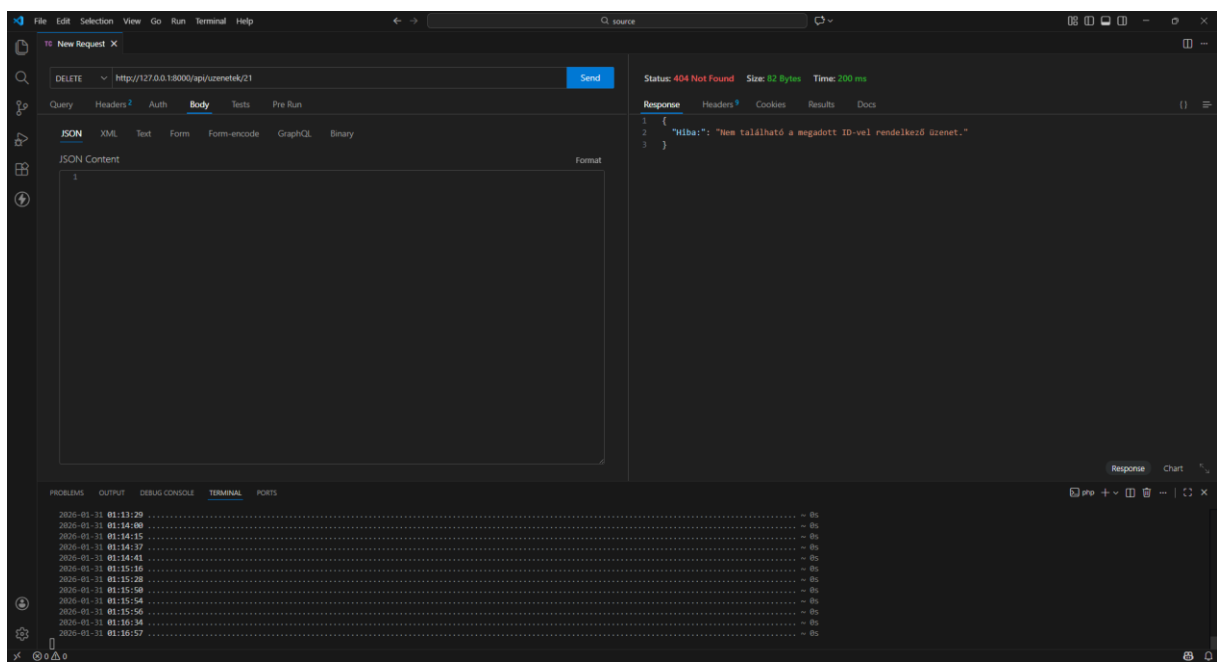
42. Kép: Get metódus sikeres teszt

üzenetek tábla Delete metódus sikeres teszt:



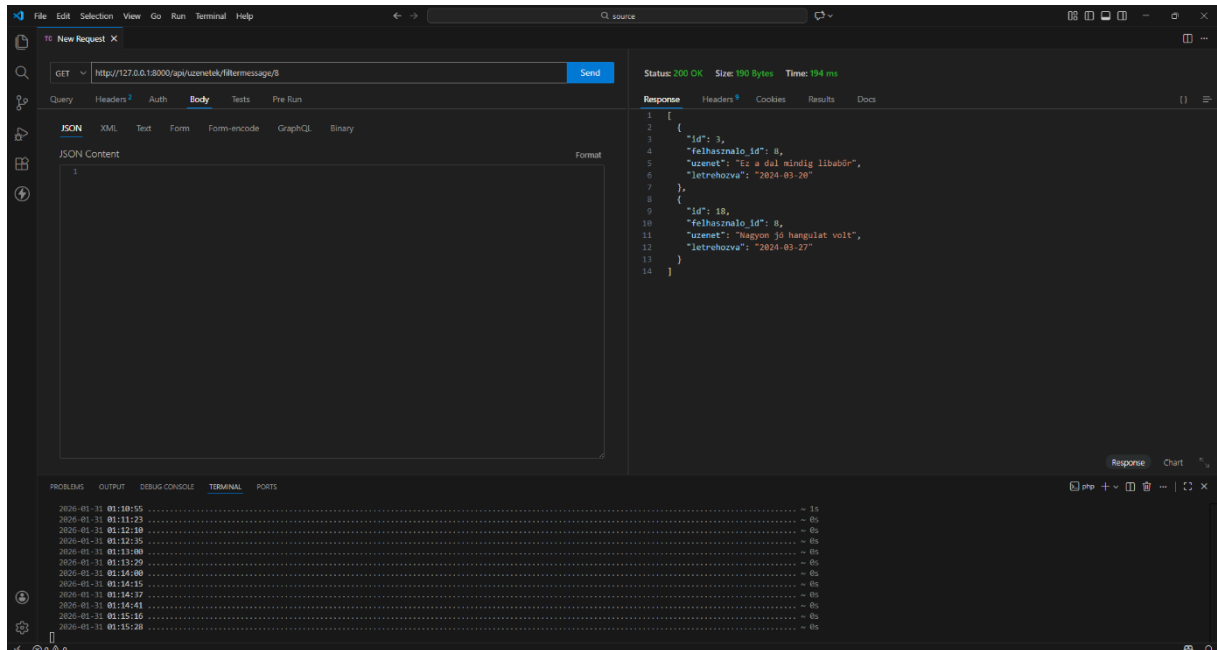
43. Kép: Delete metódus sikeres teszt

üzenetek tábla Delete metódus sikertelen teszt:



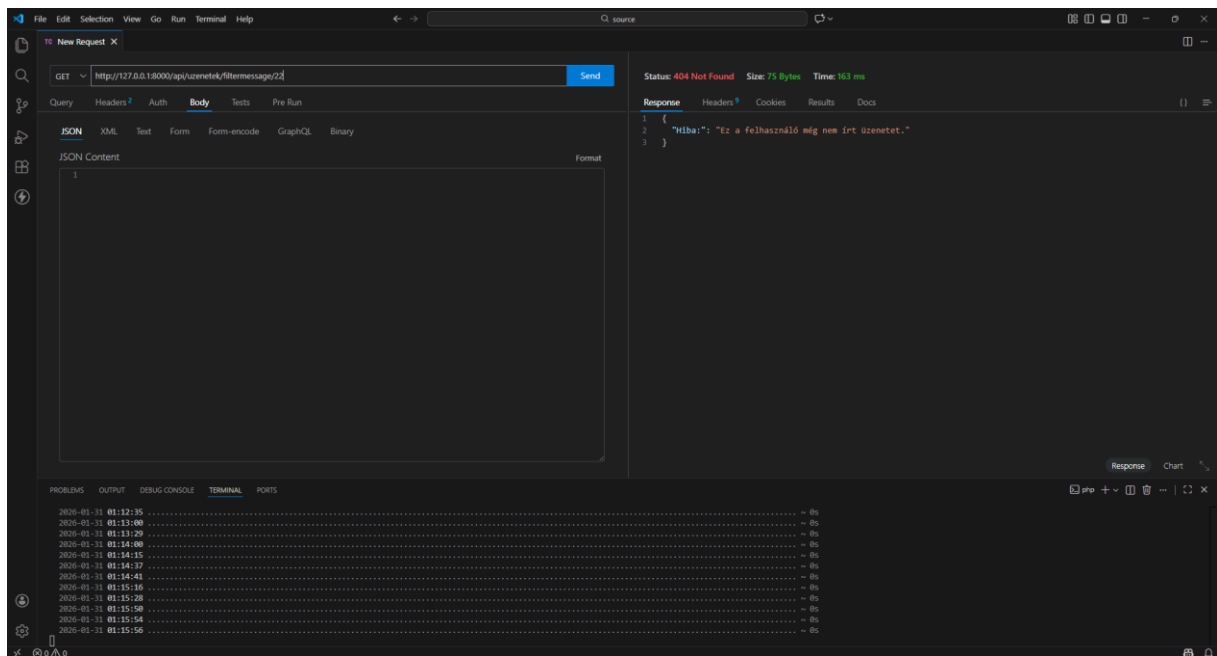
44. Kép: Delete metódus sikertelen teszt

üzenetek tábla filterBy metódus sikeres teszt:



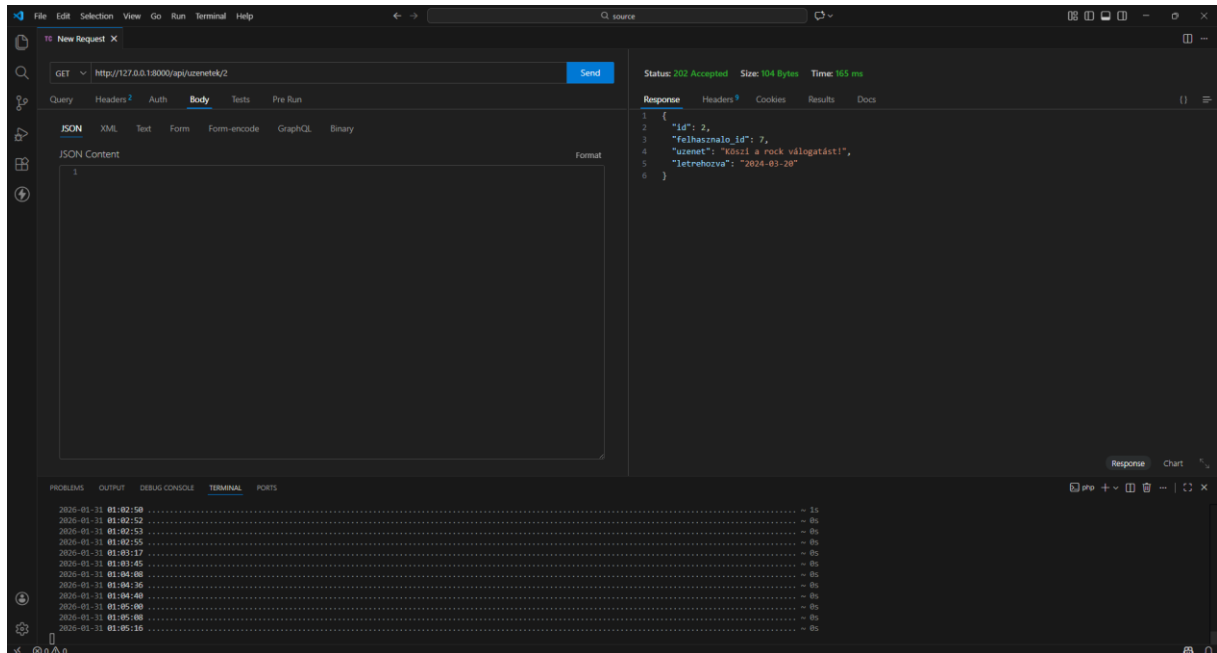
45. Kép: filterBy metódus sikeres teszt

üzenetek tábla filterBy metódus sikertelen teszt:



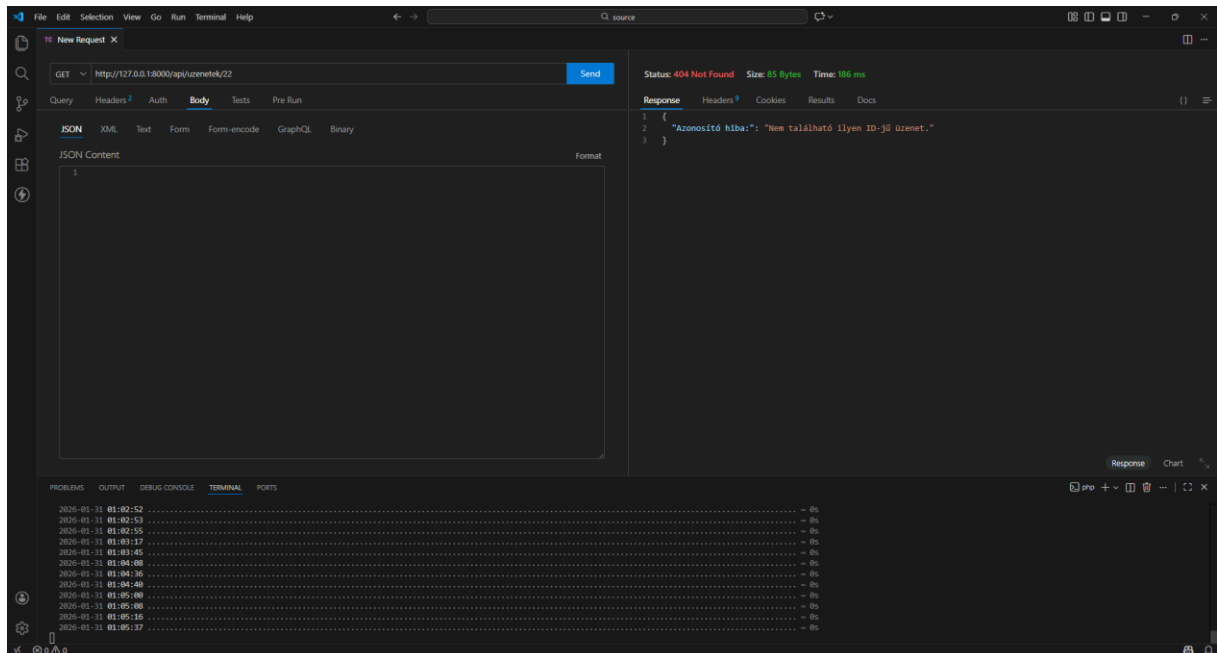
46. Kép: filterBy metódus sikertelen teszt

üzenetek tábla getByld metódus sikeres teszt:



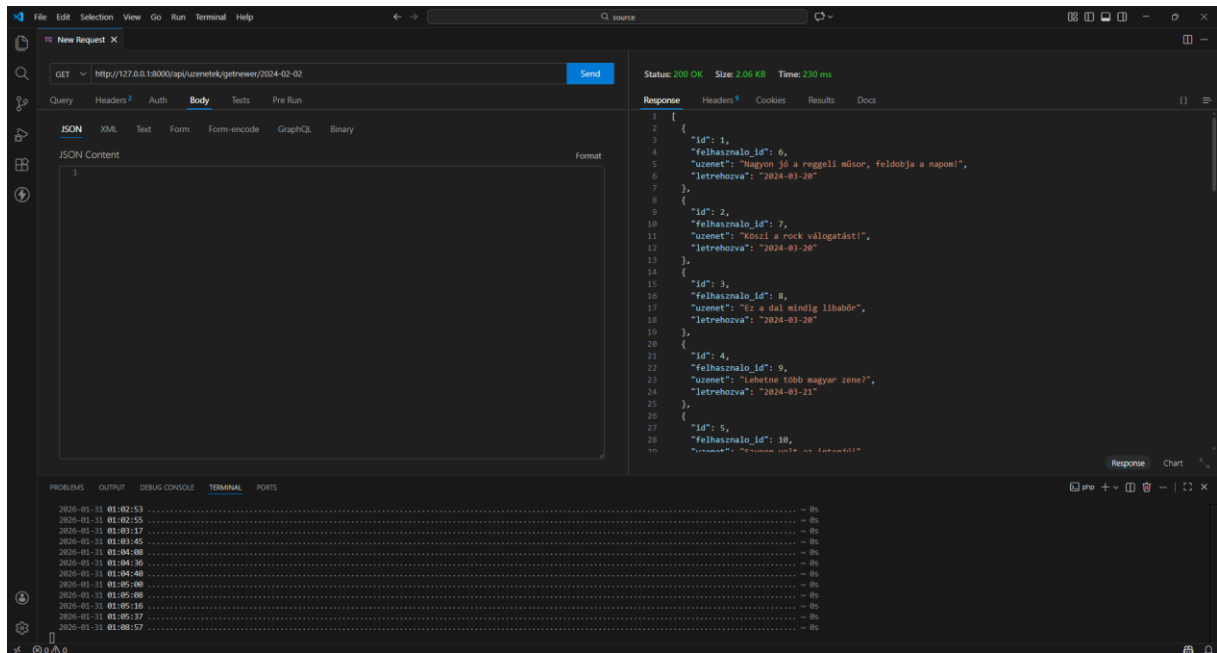
47. Kép: getByld metódus sikeres teszt

üzenetek tábla getByld metódus sikertelen teszt:



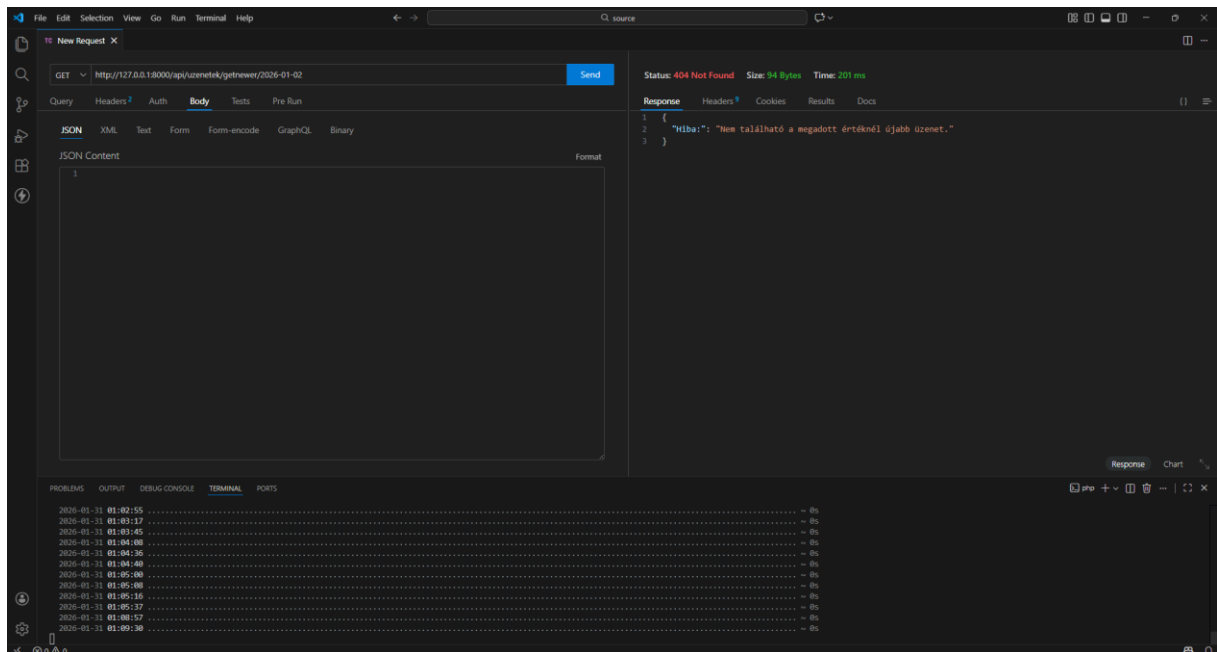
48. Kép: getByld metódus sikertelen teszt

üzenetek tábla getnewer metódus sikeres teszt:



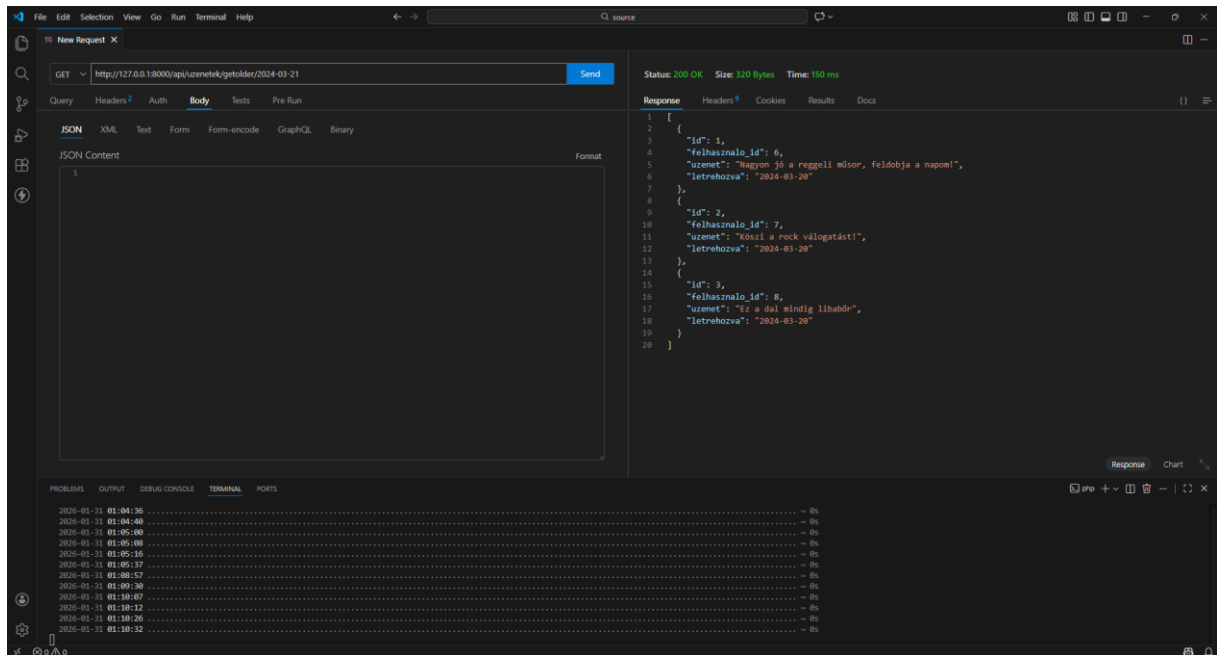
49. Kép: getNewer metódus sikeres teszt

üzenetek tábla getnewer metódus sikertelen teszt:



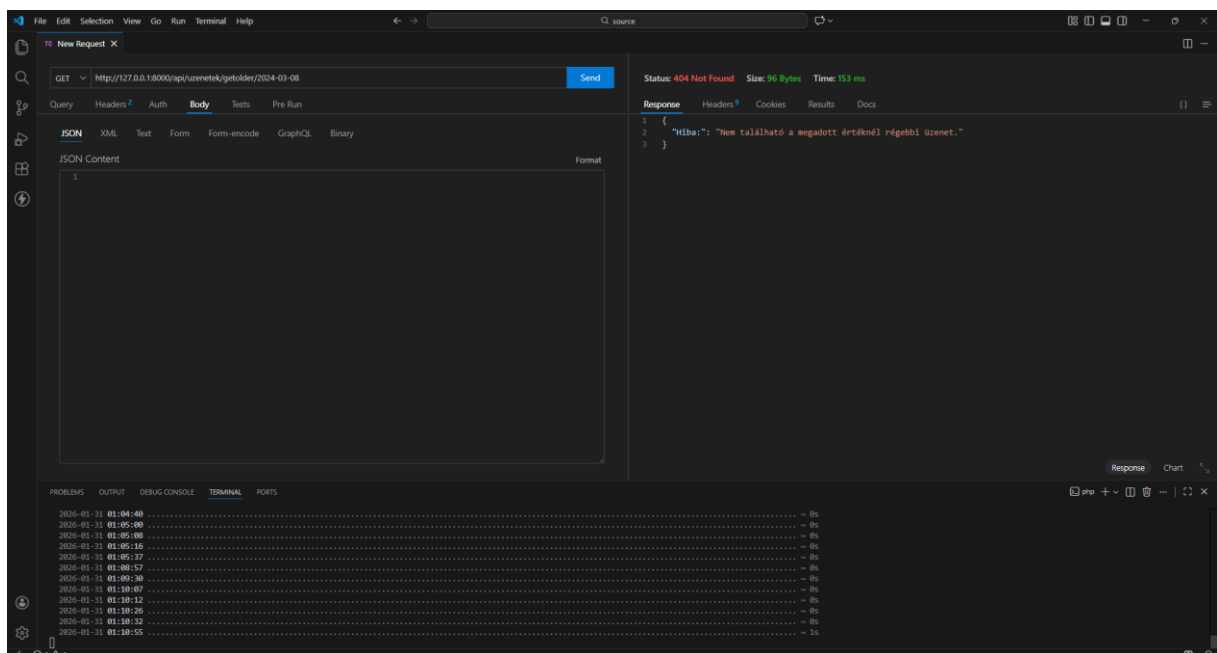
50. Kép: getNewer metódus sikertelen teszt

üzenetek tábla getolder metódus sikeres teszt:



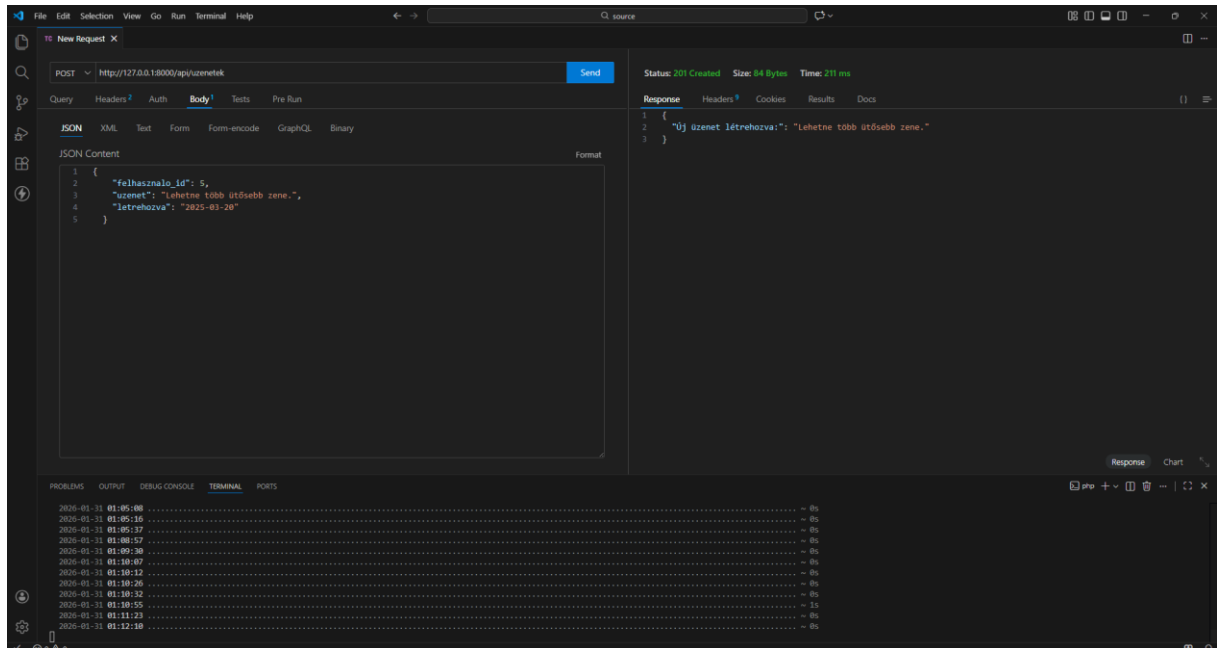
51. Kép: getOlder metódus sikeres teszt

üzenetek tábla getolder metódus sikertelen teszt:



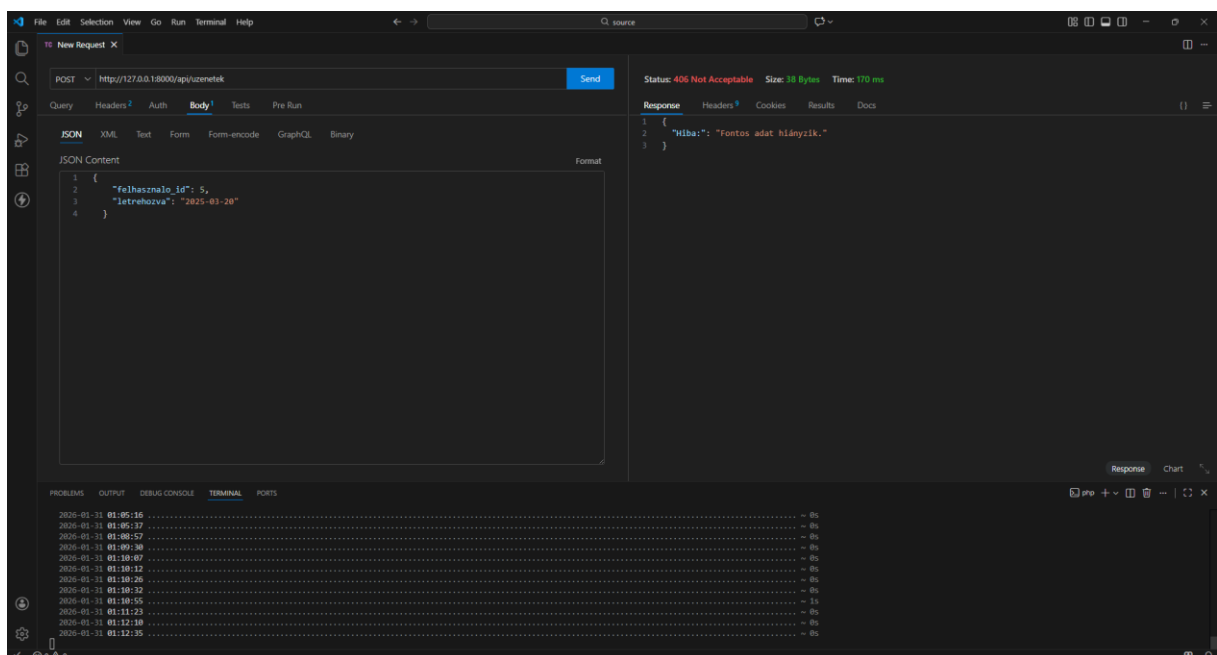
52. Kép: getOlder metódus sikertelen teszt

üzenetek tábla post metódus sikeres teszt:



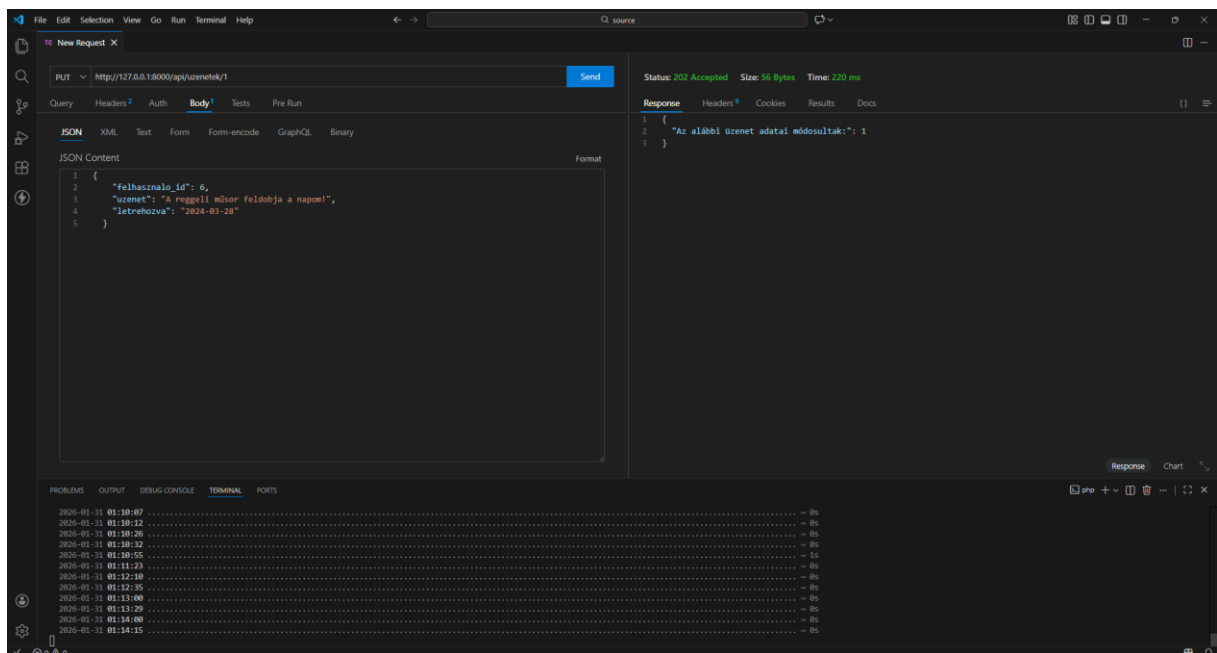
53. Kép: post metódus sikeres teszt

üzenetek tábla post metódus sikertelen teszt:



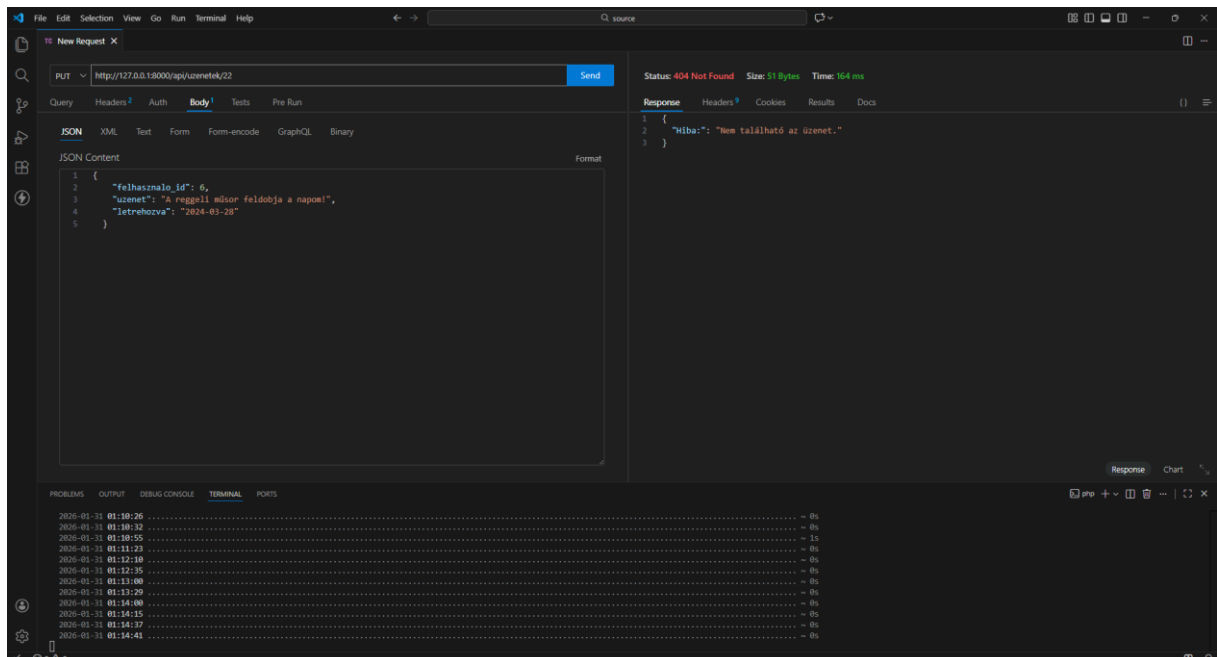
54. Kép: post metódus sikertelen teszt

üzenetek tábla put metódus sikeres teszt:



55. Kép: put metódus sikeres teszt

üzenetek tábla put metódus sikertelen teszt:



56. Kép: put metódus sikertelen teszt